

Flutter 1: Una aplicación con vistas

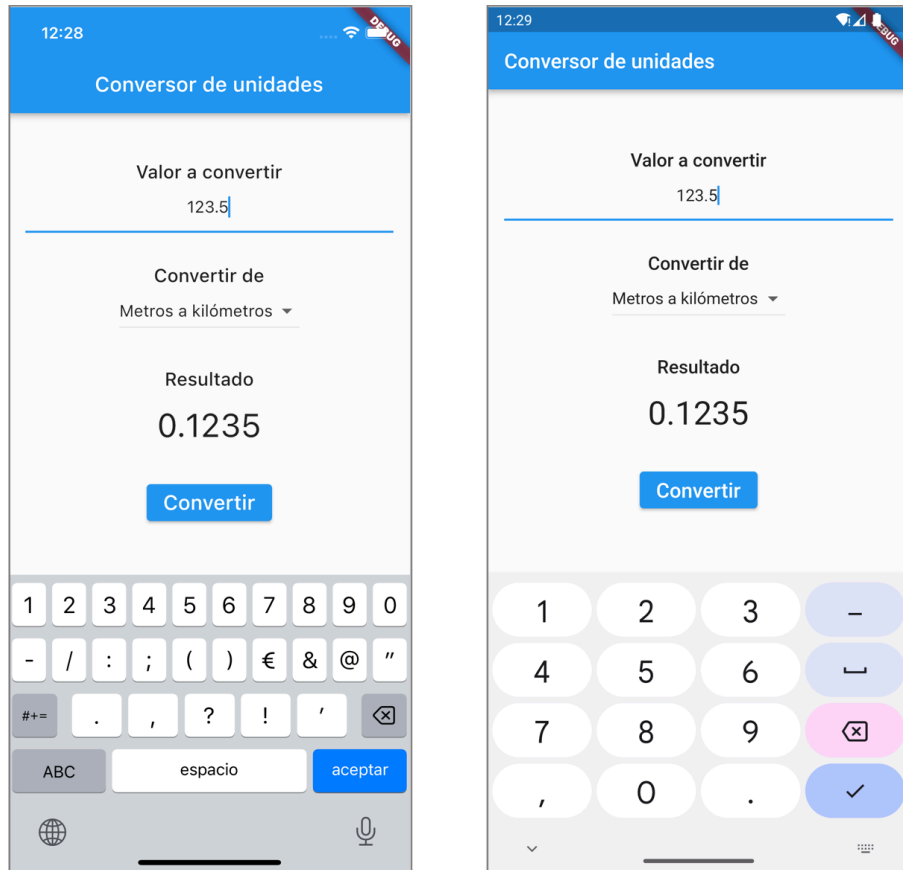
Juego de palabras con el título de la película: "Una habitación con vistas (1985)"

Contenidos

- Flutter 1: Una aplicación con vistas
 - ¿Qué voy a hacer en esta sesión?
 - Construir el esqueleto de la aplicación
 - Estructura de una aplicación Flutter
 - Ejecutar una aplicación Flutter
 - Construir nuevos emuladores en Android
 - Construir nuevos simuladores en iOS
 - Interfaz de usuario declarativa
 - Herramientas de Flutter para facilitar el desarrollo y la prueba
 - Hot Reload
 - Hot Restart
 - Flutter Structure
 - Autocomentarios
 - Escribir la aplicación conversor de unidades
 - 1 – Borrar el código de la aplicación de prueba
 - 2 – Escribir el esqueleto de la clase aplicación
 - 3 – Iniciar la ejecución de la aplicación
 - 4 – Escribir el contenido real de la clase aplicación
 - 5 – Escribir el esqueleto de la página principal de la aplicación
 - Diferencia entre widgets sin estado y widgets con estado
 - 6 – Completar la página principal de la aplicación
 - Etiqueta para el valor a convertir
 - Campo de texto donde escribir el valor a convertir
 - Añadir un desplegable para elegir el tipo de conversión
 - Medidas independientes del dispositivo
 - Densidad de píxeles de una pantalla o Screen density
 - Píxeles independientes de la densidad o Density-independent pixels
 - Añadir el botón para iniciar los cálculos
 - 7 – Probando la interfaz de usuario
 - 8 – Escribir el modelo de la aplicación
 - ¿Es realmente una buena aplicación?
 - Mini ejercicios

¿Qué voy a hacer en esta sesión?

En esta sesión escribiremos una aplicación para convertir medidas en diferentes unidades, por ejemplo: metros a kilómetros o viceversa. El aspecto de la aplicación terminada es este:



Aspecto del conversor de unidades en un emulador iOS (izquierda) y Android (derecha)

Es una aplicación sencilla, pero eso no debe engañarnos, mientras la escribimos aprenderemos:

- A manejar algunas herramientas de flutter que ahorran mucho tiempo de prueba, como **Hot Reload** y **Hot Restart**.
- Cómo es la estructura de una aplicación Flutter y que todo en Flutter son vistas (Widgets).
- A utilizar, entre otros, los widgets de Flutter: `Placeholder`, `StatelessWidget`, `StatefulWidget`, `State`, `SafeArea`, `Scaffold`, `AppBar`, `Center`, `Padding`, `Column`, `Text`, `TextField`, `DropDownButton`, `SingleChildScrollView`, `SizeBox` y `ElevatedButton`
- Cómo conseguir que las vistas se vean con tamaños similares aunque la aplicación se ejecute en dispositivos con características físicas (tamaño y resolución) muy distintas.
- Cómo se gestiona el **estado mutable** de una aplicación con una sola pantalla, y cómo se refrescan los cambios de estado en la interfaz de usuario.
- A aplicar en la práctica el principio de diseño de software **OCP** (Open Close Principle) para que la aplicación pueda soportar los cambios de forma sencilla.

Construir el esqueleto de la aplicación

Flutter permite construir una aplicación de prueba completamente funcional de una forma muy sencilla. Usar esta aplicación de prueba es la forma mas sencilla de construir la compleja estructura de carpetas y archivos que necesita una aplicación Flutter, si bien, el código de ejemplo que produce Flutter no nos será de mucha utilidad.

Para construir esta aplicación de prueba abriremos un terminal del sistema operativo y navegaremos hasta la carpeta `flutter-projects` Ahí ejecutaremos:

```
flutter create flutter_sesion_1
```

Esa orden construye los directorios y archivos necesarios para una aplicación Flutter y algunas indicaciones sobre cómo compilarla y ejecutarla.

La salida de la orden `flutter create` en un terminal de MacOS es parecida a esta:

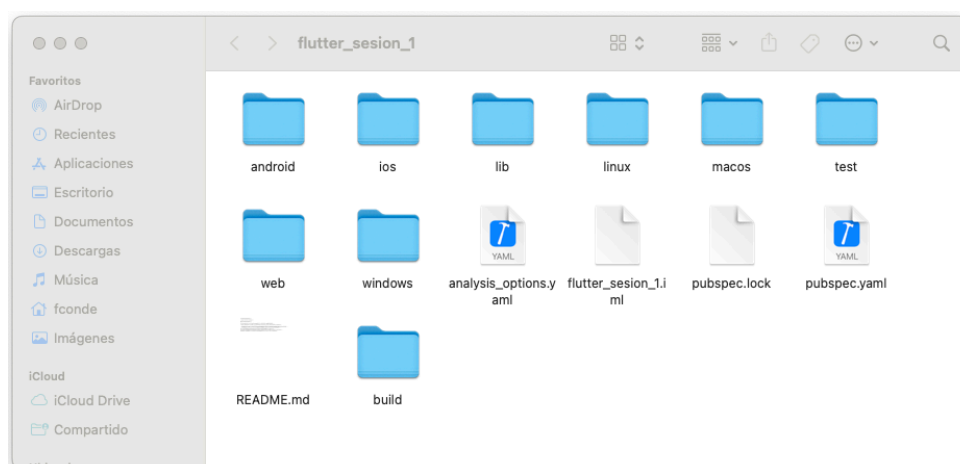
```
fconde@Mac-mini-de-Francisco flutter-projects % flutter create flutter_sesion_1
Signing iOS app for device deployment using developer identity: "Apple
Development: Francisco Conde Rodriguez"
Creating project flutter_sesion_1...
Running "flutter pub get" in flutter_sesion_1... 832ms
Wrote 127 files.

All done!
In order to run your application, type:

  $ cd flutter_sesion_1
  $ flutter run

Your application code is in flutter_sesion_1/lib/main.dart.
```

y el aspecto de la carpeta `flutter_sesion_1` es similar a este:



Aspecto de la carpeta que contiene a la aplicación en MacOS

Importante: En el nombre de un proyecto Flutter **solo se pueden usar dígitos, letras minúsculas y el símbolo de subrayado** `[a-z0-9_]`

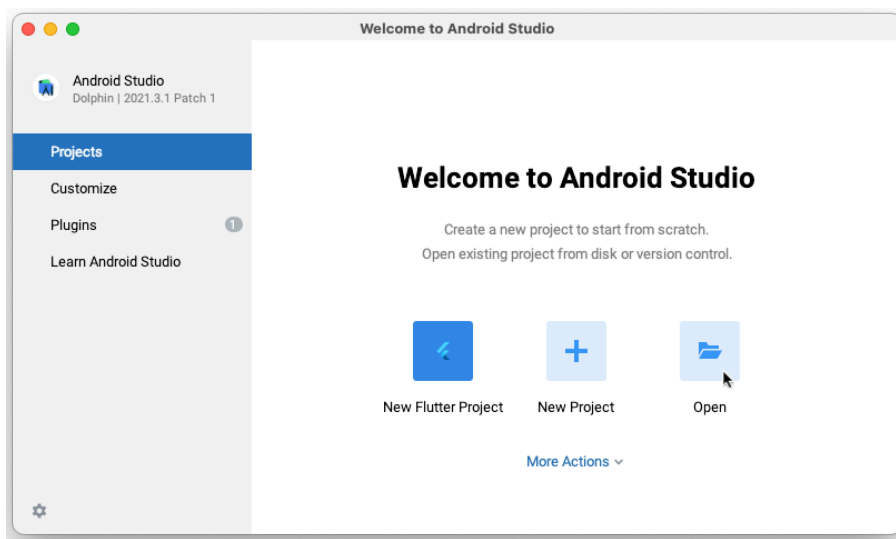
Si se escribe cualquier otro símbolo en el nombre, se produce un error. Por ejemplo, si se intenta construir una aplicación llamada `Flutter-01` se produce el error:

```
"Flutter-01" is not a valid Dart package name.  
See https://dart.dev/tools/pub/pubspec#name for more information.
```

Si ocurre algún otro error, lo más probable es que no se haya instalado correctamente Flutter y/o Android Studio o XCode (en el caso de que se esté usando un Mac).

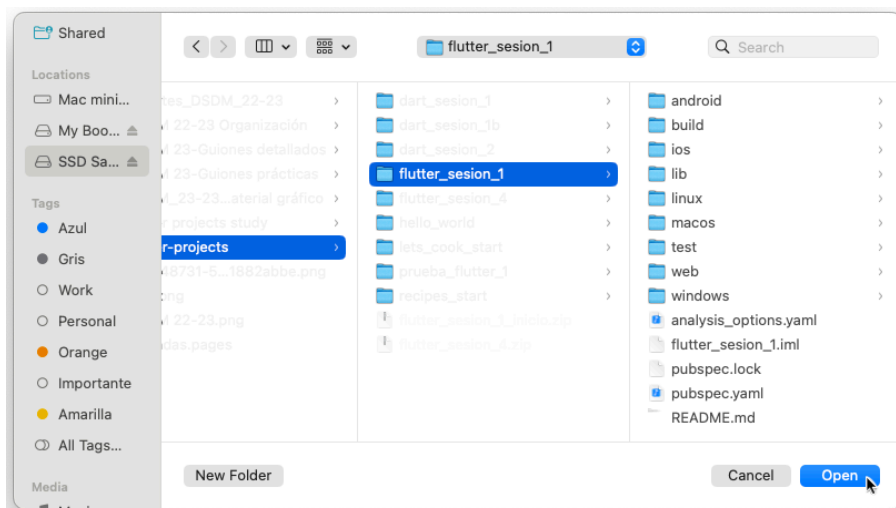
Estructura de una aplicación Flutter

Una vez creado el esqueleto de la aplicación `flutter_session_1` lo vamos a abrir con Android Studio para estudiarlo y editarlo. En el cuadro de diálogo de bienvenida de Android Studio:



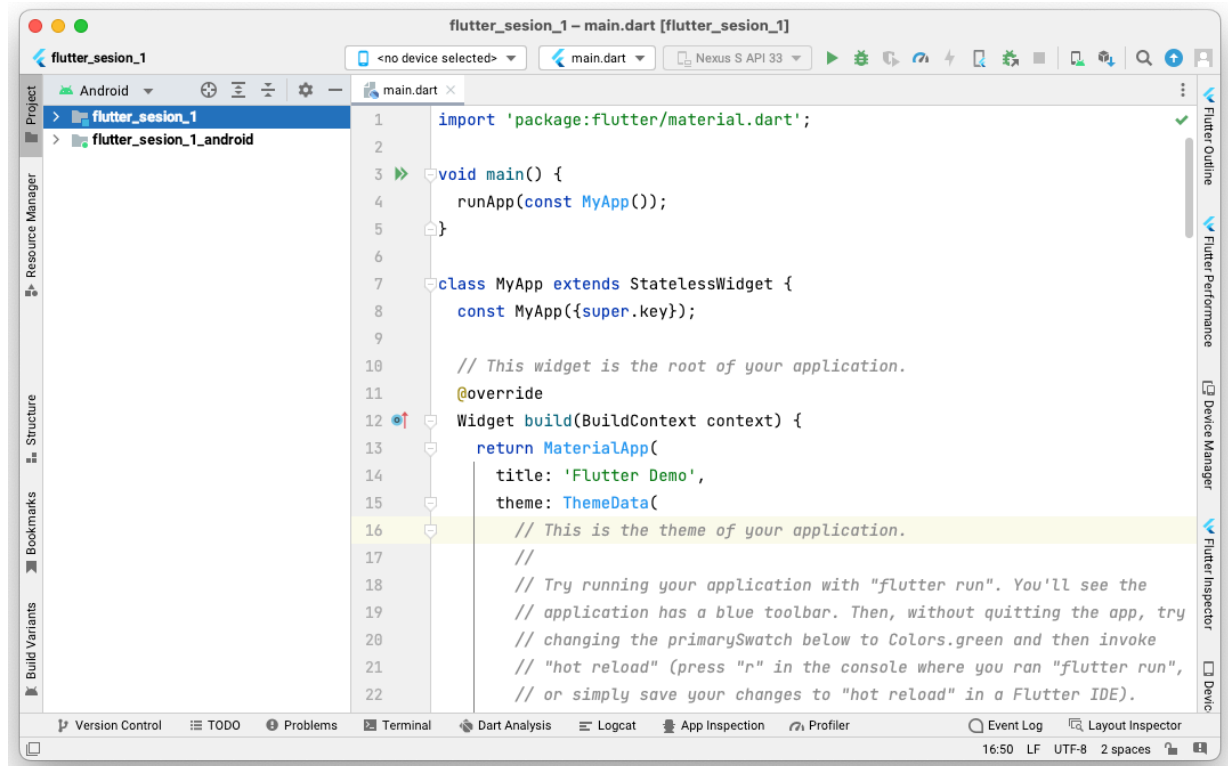
Cuadro de diálogo de inicio de Android Studio

Debemos marcar la pestaña `Projects` y en ella pulsar el botón `Open`. Se abrirá el cuadro de diálogo para abrir archivos. A continuación seleccionar la carpeta `flutter_session_1` y pulsar `Open`. El aspecto en MacOS es el siguiente:



Cuadro de diálogo para abrir un proyecto en MacOS

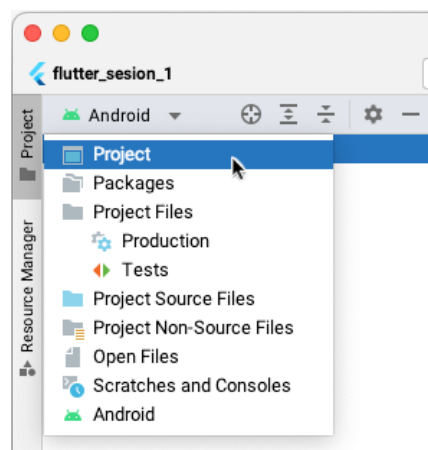
Aparece la ventana de Android Studio y en ella cargado el proyecto:



Ventana de Android Studio

Antes de ejecutarlo veamos algunas cosas importantes.

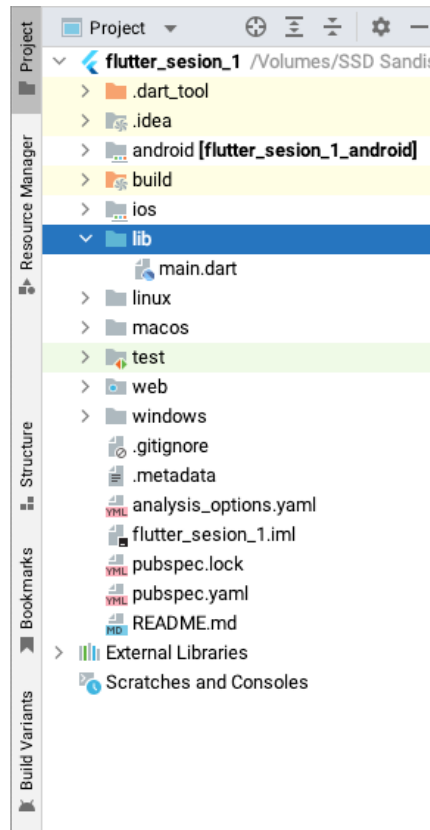
- Por defecto, Android Studio muestra la estructura del proyecto en modo **Android**. Es más cómodo mostrarla en modo **Project**. Para activar este modo, en el panel **Project** pulsar el desplegable que actualmente contiene **Android** y elegir **Project**.



Selección del modo **Project**

En este modo se muestran las carpetas que componen el proyecto de forma que los archivos que vamos a editar están más accesibles.

- En un proyecto Flutter hay una carpeta llamada **lib** que contiene los **archivos de código compartidos por todas las plataformas** y una carpeta para cada plataforma específica donde escribir el código propio de esa plataforma. Estas carpetas solo son necesarias en casos en los que se requiera alguna capacidad muy específica.



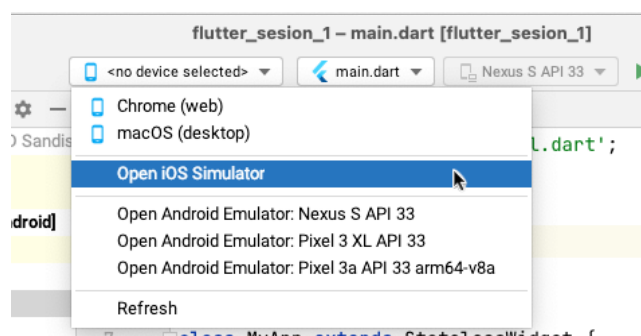
Estructura de directorios de un proyecto Flutter

Como vemos, hay carpetas con nombre `android`, `ios`, `linux`, `macos`, `web` y `windows`. Eso es porque la aplicación funciona en todas esas plataformas **sin tener que reescribir el código**.

Nota: Solo **en casos excepcionales**, en los que se requiera alguna personalización específica para alguna plataforma, haremos uso de esas carpetas.

Ejecutar una aplicación Flutter

Para ejecutar la aplicación, lo primero es arrancar algún **emulador**. Por defecto, Android Studio no arranca ningún emulador al cargar un proyecto. Para elegir un emulador se usa el desplegable etiquetado como: `<no device selected>` ▾ :

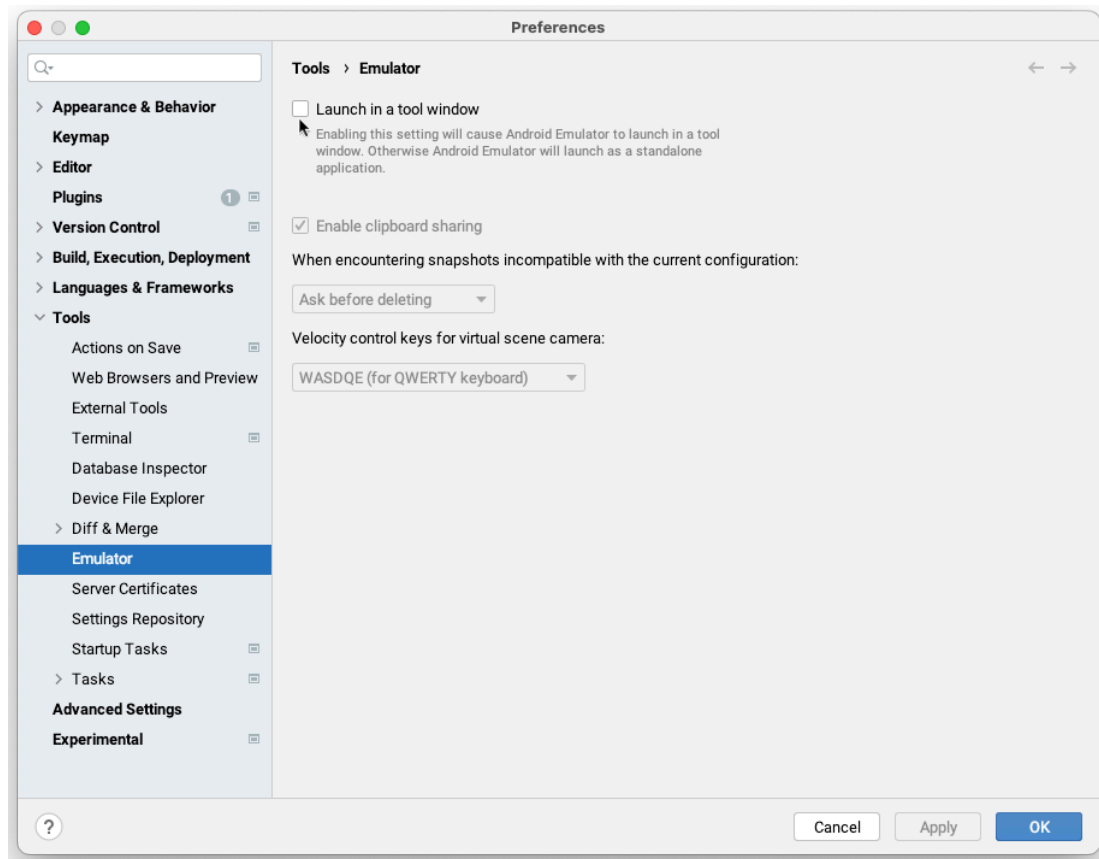


Desplegable para elegir el emulador

Nota: En muchas ocasiones, en la lista no aparecen los dispositivos disponibles directamente, sino que hay que refrescarla. Para eso se usa la última opción del menú desplegable que se llama `Refresh`

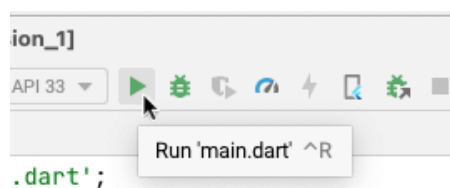
Al elegir un emulador de los que haya disponibles se ejecuta.

Nota: El emulador puede ejecutarse en un panel integrado en la ventana de Android Studio o en una ventana separada. Este comportamiento puede cambiarse abriendo el cuadro de diálogo de preferencias / ajustes de la aplicación, y en la categoría **Tools – Emulators** marcar o desmarcar (lo que se prefiera) el checkbox **Launch in a tool window**



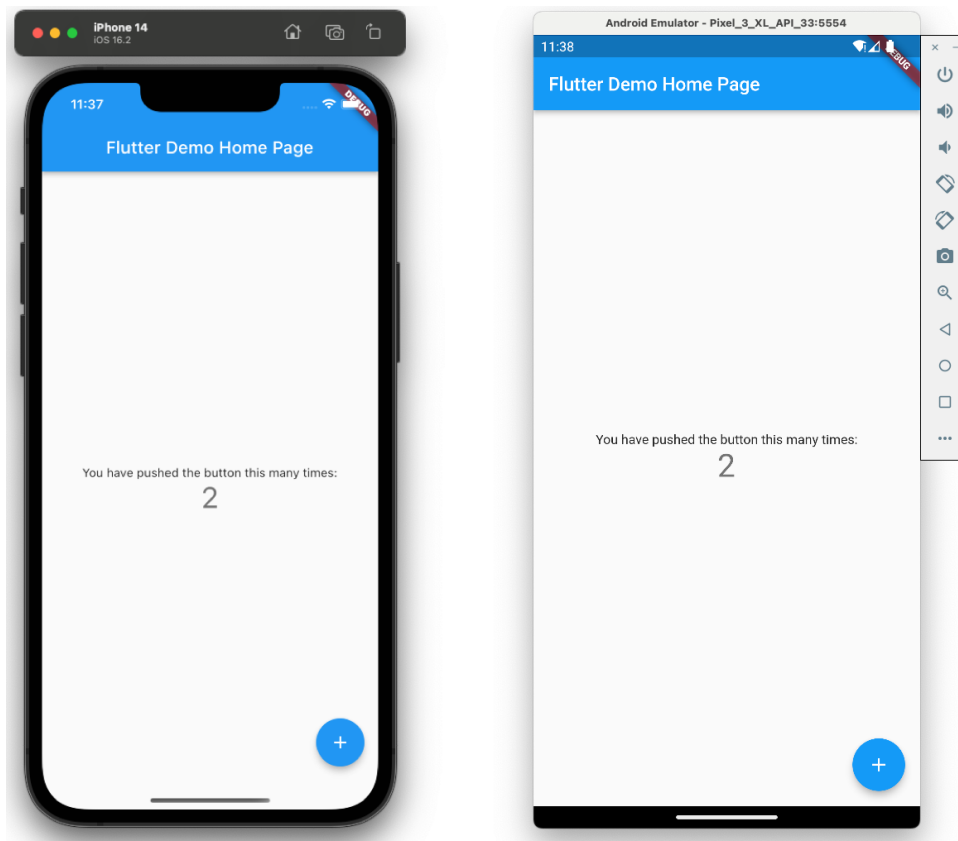
Cambiar las preferencias para el emulador

Una vez que el emulador está funcionando, se puede ejecutar la aplicación. Para ello se pulsa el icono:



Icono para ejecutar la aplicación

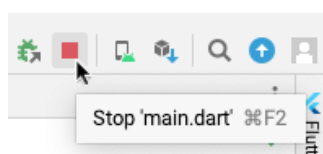
o se pulsa el atajo de teclado **^ + R** (en MacOS) o **↑ + F10** (en Windows) Eso hace que comience el proceso de compilar y enlazar la aplicación para la plataforma seleccionada (mediante el emulador elegido) y al cabo de unos segundos se muestra la aplicación en el emulador.



Aspecto de la aplicación de prueba en un emulador iOS (izquierda) y Android (derecha). En los emuladores se muestran también los controles para simular cambios de orientación, para hacer capturas de pantalla, etc.

La aplicación de prueba que construye Flutter es un contador sencillo. Pulsando en el **FAB (Floating Action Button)** azul de la esquina inferior derecha, se incrementa el valor del contador.

Para parar la ejecución de la aplicación se puede pulsar el icono:



Icono para parar la ejecución de la aplicación

o el atajo de teclado: **⌘ + F2** (en MacOS) o **^ + F2** (en Windows)

Nota: También se pueden hacer cambios en la aplicación y ejecutar esos cambios sin necesidad de recompilar todo de nuevo. Para ello se usan las funciones de Flutter **Hot Reload** y **Hot Restart**, que se estudian un poco más adelante.

Construir nuevos emuladores en Android

Nota: He preparado un vídeo con el proceso de creación de nuevos emuladores para Android:

Vídeo 2: Construir nuevos emuladores Android

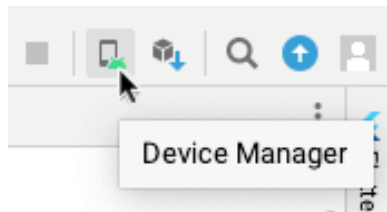
y puedes encontrarlo en la página web de la asignatura en Platea:

<https://platea.ujaen.es/mod/resource/view.php?id=186626>

En Android hay gran variedad de formatos de dispositivos y gran variedad de versiones de Android que coexisten. Si se necesita probar la aplicación en alguna concreta, hay que definirla.

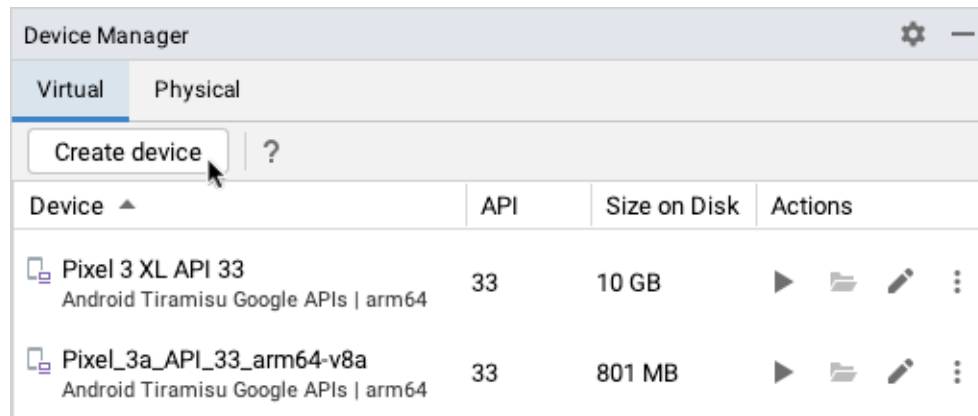
Nota: No se tienen predefinidas, porque hay muchísimas combinaciones posibles y cada una de ellas ocupa entre 500MB y 10GB.

Los emuladores en Android Studio se denominan **Virtual devices** y para gestionar los dispositivos que se tengan configurados se usa el panel **Device Manager**. Se muestra a través del menú **Tools - Device manager**, o bien mediante el icono:



Icono para acceder al panel **Device Manager**

El panel **Device Manager** tiene este aspecto:



Icono para acceder al panel **Device Manager**

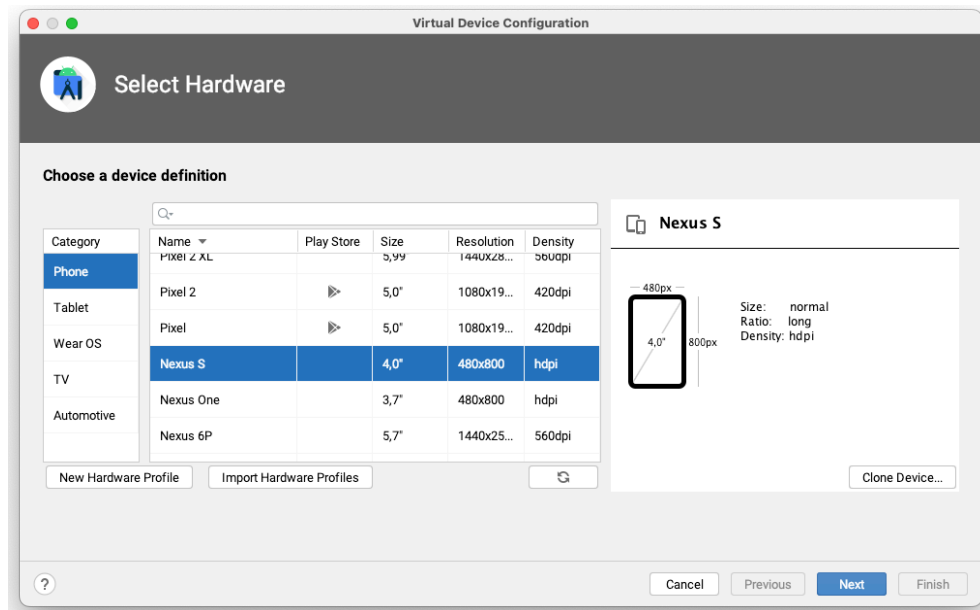
En la parte superior hay dos pestañas etiquetadas como: **Virtual** y **Physical**. En la primera se tienen los emuladores (dispositivos virtuales) y en la segunda los dispositivos físicos.

Nota: Aprenderemos a ejecutar la aplicación en dispositivos físicos más adelante.

También hay un botón etiquetado como **Create device** y una lista con los dispositivos virtuales definidos actualmente.

Pulsando el botón **Create device** podemos añadir nuevos dispositivos virtuales siguiendo los siguientes pasos:

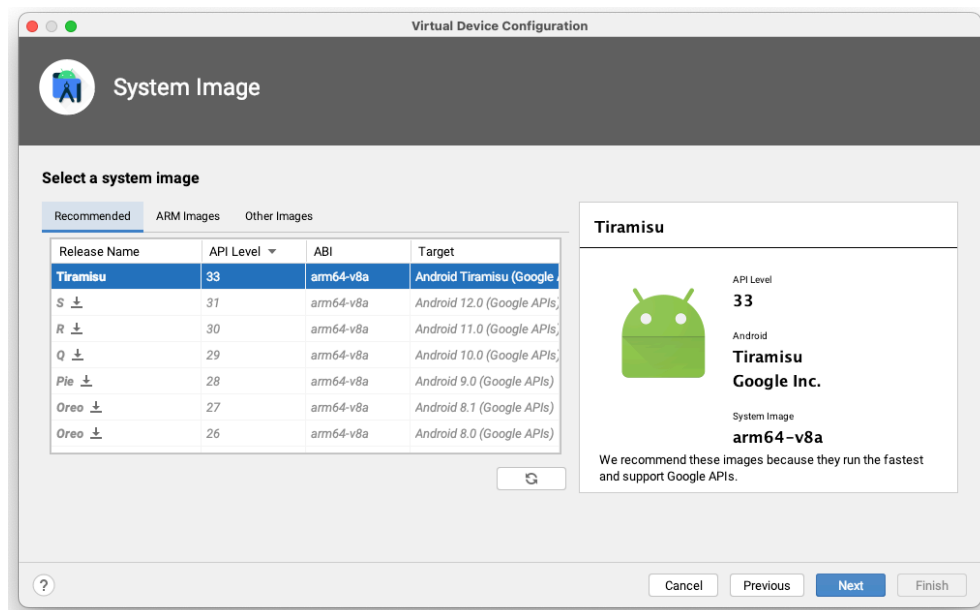
1. En primer lugar se debe escoger un **formato de dispositivo**:



Paso 1: escoger el formato de dispositivo

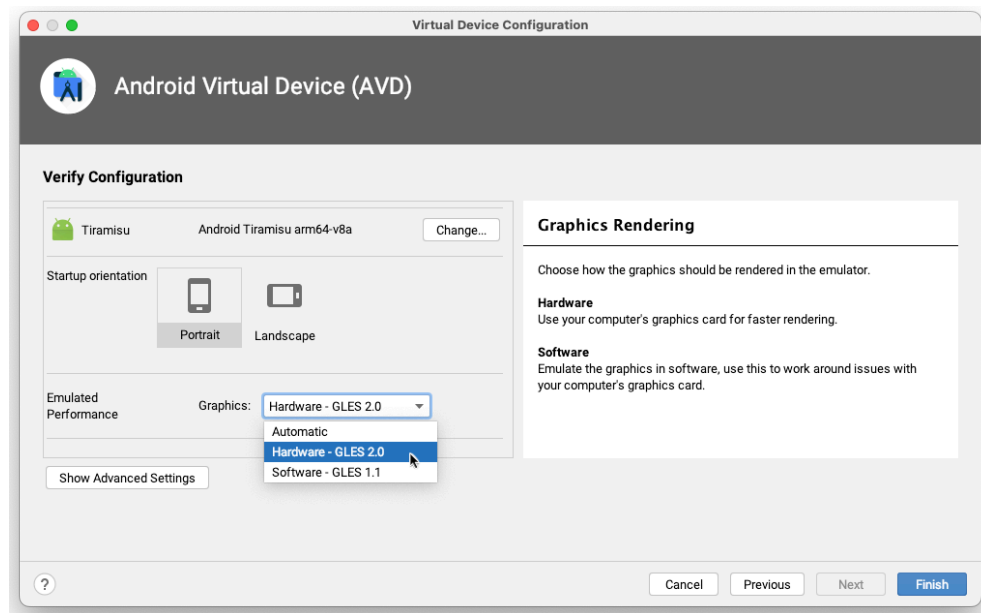
En este ejemplo, se ha escogido un Nexus S con pantalla de 4" y resolución de 480x800 píxeles.

2. A continuación se escoge la **imagen del sistema Android (versión de Android)** que se necesite:



Paso 2: escoger la versión de Android

3. Por último hay que **activar el modo:** **Hardware – GLES 2.0** en el apartado **Emulated Performance – Graphics** :



Paso 3: seleccionar el modo gráfico

Una vez hechos todos esos pasos, se pulsa el botón **Finish** y el nuevo dispositivo aparece en la lista.

Importante: En muchas ocasiones los nuevos dispositivos no aparecen inmediatamente en el menú desplegable para escoger el emulador, sino que es necesario refrescarlo pulsando **Refresh** .

Construir nuevos simuladores en iOS

En iOS, el emulador se llama **Simulator** . Una vez arrancado, se puede usar el menú: **File – Open Simulator** > para abrir simuladores para otros dispositivos Apple.

También puede usarse el cuadro de diálogo **File – New simulator...** donde se pueden elegir modelos más antiguos de iPhone y otras versiones del sistema operativo iOS (si se han instalado al configurar XCode).

Interfaz de usuario declarativa

Veamos cómo es el código de la aplicación. Para ello, en la paleta **Project** busca el archivo **main.dart** que se encuentra en la carpeta **lib** y ábrelo haciendo doble clic sobre él.

En este código hay varias cosas que llaman la atención:

1. Todas las clases de la aplicación están juntas en el mismo archivo. Esto no es bueno. Lo cambiaremos enseguida.

Nota: En Flutter, como en cualquier otra plataforma orientada a objetos, **cada clase debe ir en su propio archivo**, con la excepción de las clases privadas que trabajan estrechamente con otras clases que pueden ir juntas.

2. No hay archivo donde se defina la interfaz de usuario de forma gráfica (arrastrando y soltando componentes desde una paleta hasta un lienzo). Eso es porque en una aplicación Flutter, **la interfaz de usuario se define de forma declarativa**, es decir, sus componentes (widgets) y comportamiento se definen en código. No hay un editor gráfico para la interfaz de usuario.

Al principio puede parecer que de esta forma es más difícil construir la interfaz de usuario, pero es justo al contrario, poder declarar la interfaz en código hace que el proceso sea mucho más flexible como veremos a lo largo de estas sesiones.

Por esa razón en Flutter, así como en otras plataformas modernas como SwiftUI de Apple, **la interfaz se declara en código**.

Herramientas de Flutter para facilitar el desarrollo y la prueba

Flutter proporciona varias ayudas para escribir de forma más sencilla la declaración de los widgets que componen la interfaz de usuario de una aplicación.

Nota: He preparado un vídeo donde describo los procesos **Hot Reload** y **Hot Restart**. El vídeo se llama:

Vídeo 3: Hot reload y hot restart

y puedes encontrarlos en la página web de la asignatura en Platea:

<https://platea.ujaen.es/mod/resource/view.php?id=186627>

Hot Reload

En Dart se pueden hacer cambios que no afecten profundamente a la estructura de vistas de la aplicación y verlos reflejarse en la interfaz de usuario de inmediato **sin necesidad de recompilar la aplicación**. Esto se denomina **hot reload** y ahorra mucho tiempo de pruebas.

Usar Hot Reload es sencillo, si se tiene **una aplicación en ejecución en un emulador**, sólo hay que cambiar lo que se necesite en el código y guardar los cambios en disco (en MacOS con el atajo de teclado `⌘ + S` y en Windows con `^ + S`)

Automáticamente los cambios realizados se reflejan en la aplicación en ejecución.

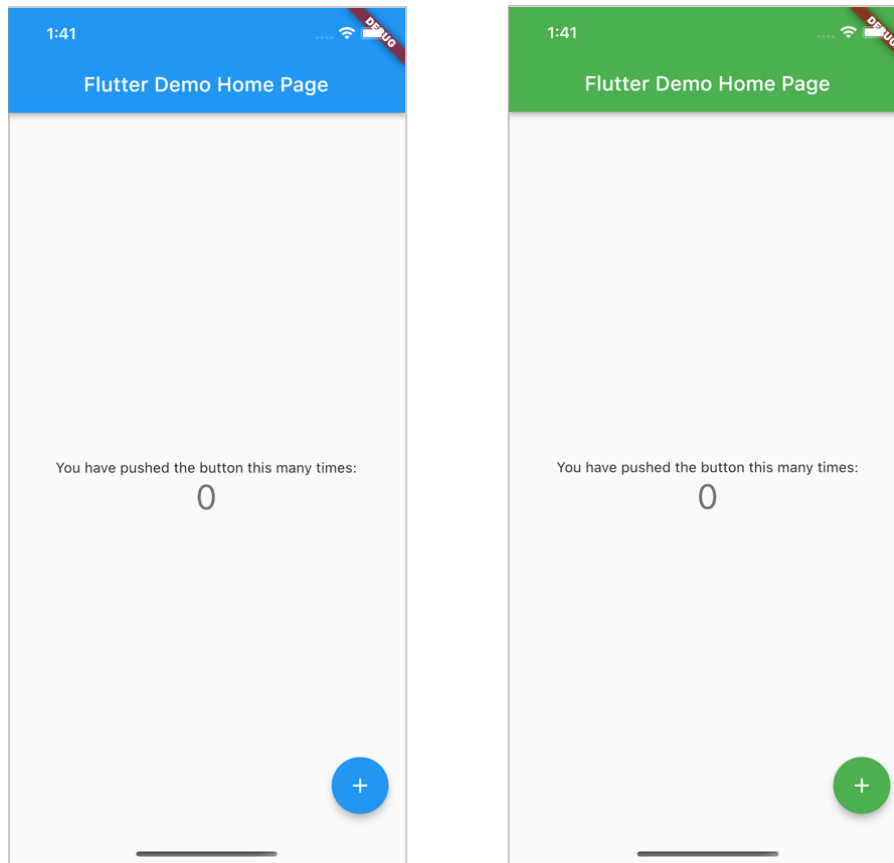
Para probarlo busca, en el archivo `main.dart` el método `build` de la clase `MyApp` y en él la línea:

```
primarySwatch: Colors.blue,
```

borra la palabra `blue` y escribe en su lugar otro color, por ejemplo `green` :

```
primarySwatch: Colors.green,
```

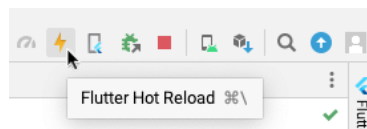
A continuación pulsa `⌘ + S` (en MacOS) o `^ + S` (en Windows) para salvar el archivo.



Aplicación de prueba de Flutter antes (izquierda) y después (derecha) de hacer **Hot Reload**

Como puedes ver, **la interfaz cambia de inmediato mientras se sigue ejecutando la aplicación.**

Nota: También se puede hacer Hot Reload pulsando el icono:



Botón **Hot Reload**

o su atajo de teclado `⌘ + \` (en MacOS) o `^ + \` (en Windows)

Hot Restart

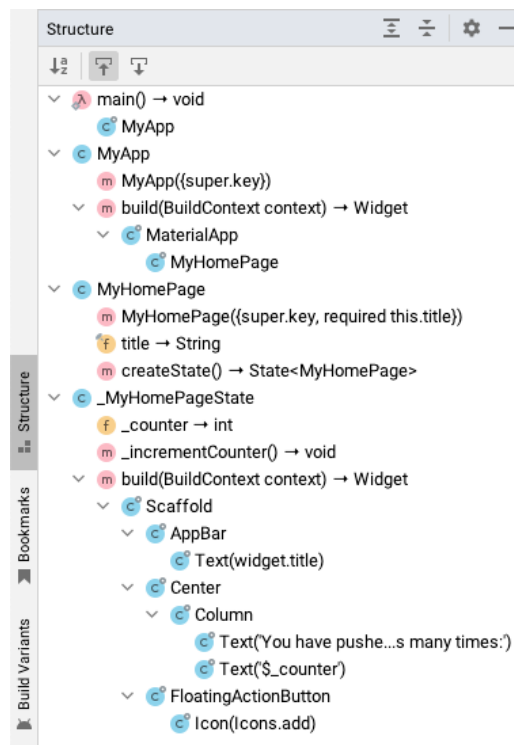
Si los cambios son profundos, Hot Reload no produce el refresco de los cambios. En lugar de eso se puede hacer **Hot Restart**. Para ello, **sin haber detenido el funcionamiento de la aplicación**, volvemos a pulsar el botón  o su atajo de teclado: `⌘ + R` (en MacOS) o `⇧ + F10` (en Windows).

El proceso de Hot Restart es un poco más complejo, pero no tanto como volver a compilar y arrancar la aplicación, por lo que también es una buena opción para probar.

Si tampoco funciona Hot Restart, entonces hay que detener la aplicación mediante el icono  (cuadrado rojo), cuyo atajo de teclado es `⌘ + F2` (en MacOS) o `^ + F2` (en Windows) y a continuación volver a pulsar el botón  o su atajo de teclado: `⌘ + R` (en MacOS) o `⇧ + F10` (en Windows). Eso produce que se recompila la aplicación antes de volver a arrancarla.

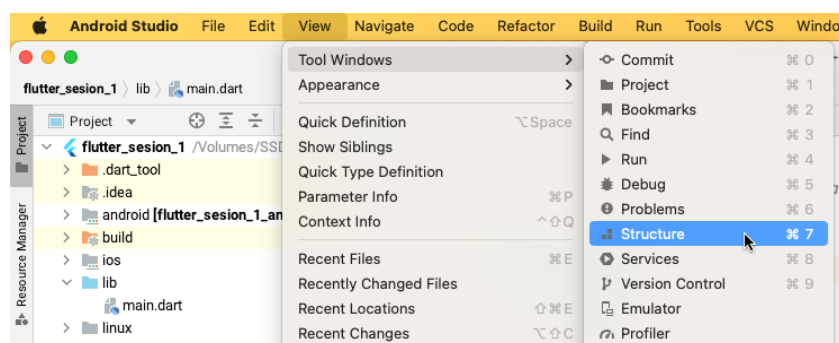
Flutter Structure

Una ayuda visual muy importantes es **Flutter Structure** que es una **representación visual del árbol de widgets** de una aplicación Flutter:



Aspecto de la paleta **Structure** donde se muestra el árbol de widget de la aplicación de prueba

Se muestra en la paleta **Structure** que puede mostrarse y ocultarse mediante el menú: **View -> Tool Windows -> Structure**



Menú para mostrar / ocultar la paleta **Structure**

En la paleta **Structure** hay nodos de color azul claro. Son los distintos widgets del árbol. También hay nodos en color rosa que corresponden con los métodos que declaran esos widgets, y nodos de color amarillo que corresponden con las propiedades importantes de esos widgets. Así, de un vistazo, se puede comprender mejor la estructura de vistas de una aplicación Flutter.

Una ventaja de la paleta **Structure** es que si se hace clic sobre alguno de los nodos del árbol, el editor muestra el código donde se declara ese nodo, de forma automática. Eso facilita la búsqueda de un nodo para su edición.

Consejo: Encontrar un widget, alguna de sus propiedades o algún método, buscándolo en el código puede ser difícil, sobre todo, cuando se tiene una interfaz de usuario compleja con muchos componentes. Es más sencillo localizarlo en el árbol de widgets y hacer clic ahí para que se muestre automáticamente su código asociado.

Autocomentarios

Otra ayuda importante para entender el código de Flutter son los **autocomentarios**. Son comentarios en formato: `// [tipo_widget]` que **Flutter genera automáticamente**, que **no se pueden editar ni borrar** y que ayudan a identificar la extensión de la definición de los distintos componentes de la interfaz.

En la siguiente imagen puede verse los paréntesis y corchetes de cierre de algunos widgets de la aplicación de prueba de Flutter. Como puede verse, junto a ellos, Flutter escribe un autocomentario indicando a qué widget corresponden. De esta forma es más sencillo entender en código de Flutter.



Autocomentarios de algunos widgets de la aplicación de prueba de Flutter

Escribir la aplicación conversor de unidades

Podríamos intentar reutilizar el código de la aplicación de prueba para escribir el código de nuestra aplicación conversor, pero vamos a escribirlo desde cero para poder aprender mejor algunas cosas fundamentales de las aplicaciones Flutter.

1 – Borrar el código de la aplicación de prueba

Comenzaremos borrando todo el contenido del archivo `main.dart` y sustituyéndolo por una función `main` vacía:

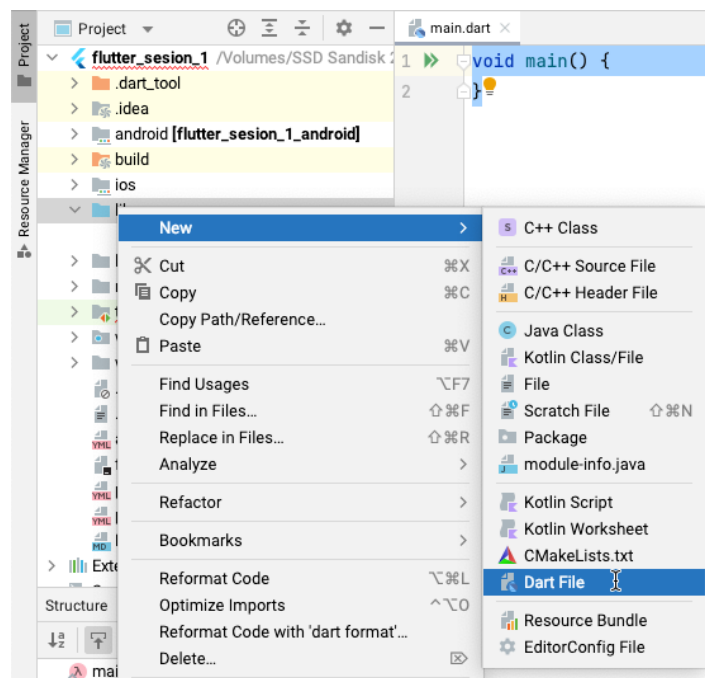
```
void main() {
}
```

La función `main` es muy importante porque, como en otros lenguajes de programación, es el **punto de partida de la aplicación**. Cuando se ejecuta la aplicación se busca esta función y se ejecuta.

2 – Escribir el esqueleto de la clase aplicación

El siguiente paso es comenzar a escribir la clase aplicación. Será ésta la clase que se llamará dentro de la función `main` para iniciar la ejecución de la aplicación.

Como sabemos, cada clase en Flutter debe ir en su propio archivo, así que empezaremos añadiendo un nuevo archivo llamado `conversor_app.dart`. Para eso, en la pestaña **Project** de Android Studio, haremos clic derecho sobre la carpeta `lib` para que aparezca el menú de contexto y ejecutaremos el menú `New > – Dart File`.

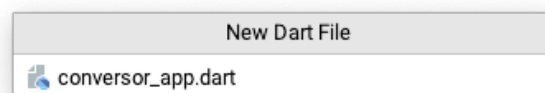


Menú para añadir nuevos archivos a la aplicación

Aparece un pequeño cuadro de diálogo en el que escribiremos el nombre del archivo:

`conversor_app.dart`

y pulsaremos `Enter`



Cuadro de diálogo para escribir el nuevo nombre del archivo

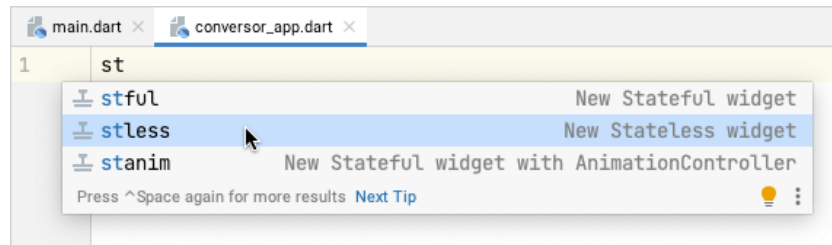
El nuevo archivo aparece dentro de la carpeta `lib` junto al archivo existente `main.dart`

Nota: No es obligatorio seguir, para los nombres de archivo, el mismo convenio que se sigue para el nombre de la aplicación, pero Google lo recomienda, así que es el criterio que seguiremos en estas prácticas para nombrar los archivos de código.

Dentro de ese archivo declararemos la clase `ConversorApp`. La forma más sencilla de hacerlo en Android Studio es comenzar a escribir:

```
stless
```

y cuando aparezca la sugerencia de código pulsar `Enter`. A continuación teclear `ConversorApp` y pulsar de nuevo `Enter`.



Sugerencia de código para escribir subclases de `StatelessWidget`

Aparece un fragmento de código con la declaración de una nueva clase en Dart. Ese código tiene algunos errores de compilación que se deben a que falta **importar un paquete**, así que en la primera línea del archivo escribiremos:

```
import 'package:flutter/material.dart';
```

En este punto, el contenido del archivo `conversor_app.dart` debería ser:

```
import 'package:flutter/material.dart';

class ConversorApp extends StatelessWidget {
  const ConversorApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return const Placeholder();
  }
}
```

Antes de seguir, veamos algunos aspectos importantes:

1. Un **paquete** es una biblioteca que contiene código reutilizable. Para poder usarlo hay que importarlo mediante la instrucción `import` seguida del nombre del paquete entre comillas simples `' '`.

El paquete `flutter/material.dart` define widgets con los que construir la interfaz de usuario. En concreto proporciona *widgets de tipo material* que implementan **Material Design**, el lenguaje visual de descripción de interfaces desarrollado por Google.

Flutter proporciona cientos de paquetes que nos permiten construir interfaces de usuario muy avanzadas con poco esfuerzo.

2. La clase `ConversorApp` es una subclase de `StatelessWidget` que como su nombre indica **¡es un widget!**

Importante: Todo en Flutter son vistas, hasta la propia aplicación es una vista.

Estamos hablando de **vistas** y **widgets** como si fueran lo mismo, pero no lo son aunque están muy relacionadas. Un widget es una descripción de un componente de la interfaz de usuario. Esta descripción se convierte en una vista física que se dibuja en la pantalla, cuando se construye la interfaz, al arrancar la aplicación.

La interfaz de una aplicación Flutter se describe mediante un **árbol de widgets**, en el que unos widgets contienen a otros. Ese árbol de widgets se transformará en un **árbol de vistas** al ejecutar la aplicación.

3. Todas las subclases de `StatelessWidget` tienen que redefinir el método:

```
Widget build(BuildContext context)
```

que como vemos devuelve un objeto de clase `Widget` que es la superclase de todas las vistas que se pueden usar en Flutter. Este método se llama cuando el `StatelessWidget` se inserta en el árbol de widgets. Su misión es describir mediante código cómo es la parte de la interfaz de usuario que está contenida dentro de la vista.

El código que nos ha proporcionado Flutter devuelve una instancia de `Placeholder` que es un widget que dibuja un cuadro que representa dónde se situarán otros widgets en el futuro.

`Placeholder` es un widget útil durante el desarrollo, para indicar que la interfaz de usuario aún no está completa.

El método `build` siempre recibe como único argumento el contexto de la aplicación. Este contexto es compartido por todos los widgets del árbol de widgets. A través de él se puede acceder, por ejemplo, al tema general de la aplicación (define los colores, tipos de letra, etc. que se usan en toda la aplicación) y a otra información útil que veremos en próximas sesiones.

4. El constructor de `ConversorApp` es `const` lo que como sabemos, significa que se construye una **instancia canónica de la clase**. Es decir, aunque se construyan muchas instancias de la clase `ConversorApp` todas ellas son la misma instancia en memoria.

Eso es así ya que una `StatelessWidget` es **inmutable**. Una vez que se crea, no puede cambiarse, o lo que es igual, se crea solo una vez, y no se puede volver a reconstruir durante la ejecución de la aplicación. El método `build()` de una subclase de `StatelessWidget` se llama solo una vez.

Esto es muy importante, ya que cada vez que se refresque la interfaz de usuario de la aplicación se debe reconstruir el árbol de vistas. Al tener una instancia canónica, solo se construye la instancia de `ConversorApp` la primera vez, el resto de veces que se refresca la interfaz se reutiliza la misma instancia, con el consiguiente ahorro de recursos.

Importante: Para que nuestra aplicación móvil sea realmente eficiente, todas aquellas vistas que no cambien durante la ejecución de una aplicación como por ejemplo, títulos o botones, deberían declararse como `const`. De esta forma no se generarán nuevas instancias de esas vistas que no cambian al refrescar la interfaz de usuario.

3 – Iniciar la ejecución de la aplicación

Hasta ahora, tenemos una función `main` vacía en el archivo `main.dart` y una clase aplicación `ConversorApp` en el archivo `conversor_app.dart`. Es el momento de conectar ambas cosas para que al iniciar la aplicación se cargue la clase `ConversorApp` que como sabemos es el nodo raíz del árbol de widgets de la aplicación.

Para ello, en el archivo `main.dart`, dentro del cuerpo de la función `main` escribiremos:

```
runApp(const ConversorApp());
```

y al comienzo del archivo `main.dart` escribiremos los `import` necesarios para que no haya errores de compilación:

```
import 'package:flutter/material.dart';
import 'package:flutter_sesion_1/conversor_app.dart';
```

con lo que el contenido del archivo `main.dart` completo debería ser:

```
import 'package:flutter/material.dart';
import 'package:flutter_sesion_1/conversor_app.dart';

void main() {
  runApp(const ConversorApp());
}
```

Veamos algunos aspectos importantes:

1. El código de la función `main` es muy simple: se encarga de arrancar la ejecución de la aplicación mediante el método `runApp` que recibe un objeto de clase `ConversorApp`.

El método `runApp` recibe como argumento un widget que contiene otros widgets y los transforma en las distintas vistas de la interfaz de usuario.

2. El argumento que se pasa a `runApp` es `const` ya que la clase `ConversorApp` tiene un constructor `const`.

Podríamos no escribir la palabra `const` delante de la llamada al constructor de `ConversorApp` pero el compilador producirá el siguiente warning:

```
PREFER using const for instantiating constant constructors.
```

donde se nos avisa que si se puede invocar un constructor como `const` para producir una instancia canónica, es preferible hacerlo, ya que de esta forma la aplicación es mucho más eficiente.

3. Dado que la función `main` solo tiene una línea de código, podría haberse escrito usando notación flecha así:

```
void main() => runApp(const ConverterApp());
```

4 – Escribir el contenido real de la clase aplicación

La clase `Placeholder` es útil para poder compilar la aplicación cuando no está completa la interfaz, pero no es el contenido que queremos para la clase `ConverterApp`. Lo vamos a cambiar. En el archivo `converter_app.dart` buscaremos la línea:

```
return const Placeholder();
```

y la sustituiremos por:

```
return MaterialApp(  
  title: 'Converter',  
  theme: ThemeData(  
    primarySwatch: Colors.blue,  
  ),  
  home: const Placeholder(),  
);
```

Veamos algunos aspectos importantes de ese código antes de seguir:

1. Todas las aplicaciones en Flutter devuelven un objeto de clase `MaterialApp` que es el contenedor que usaremos para construir interfaces de usuario basadas en Material Design. Se usa el constructor generativo personalizado de `MaterialApp` que tiene muchos parámetros con nombre (algunos opcionales y otros no) que permiten personalizar su apariencia y comportamiento.

En este ejemplo se han usado tres:

- i. `title`: que es el título de la aplicación. En este ejemplo el título de la aplicación es:

```
title: 'Converter',
```

- ii. `theme`: que permite elegir el *tema general* de la aplicación. Define la apariencia global de las vistas que la forman. Define cómo se van a visualizar las demás vistas de la aplicación. En este ejemplo, utiliza el color azul para la barra de título.

```
theme: ThemeData(  
  primarySwatch: Colors.blue,  
),
```

Nota: En la próxima sesión de prácticas aprenderemos a escribir y usar un tema predeterminado completo.

- iii. `home:` que recibe como argumento otro `Widget` que es la **página principal de la aplicación**. En este caso, dado que aún no hemos escrito el código de esa página principal, de nuevo se usa la clase `Placeholder`

La instancia de `Placeholder` que se pasa al parámetro `home:` se ha declarado como `const` de esta forma no hay que reconstruirla cada vez que se deba refrescar la interfaz de usuario con el consiguiente ahorro de recursos.

5 – Escribir el esqueleto de la página principal de la aplicación

Como hemos visto, la propiedad `home:` de la clase `MaterialApp` espera que se le pase un widget que es la página principal de la aplicación. En este paso vamos a construirlo. Para ello añadiremos un nuevo archivo a la carpeta `lib` llamado `conversor_pagina_principal.dart`

En él añadiremos la clase `ConversorPaginaPrincipal` que en este caso es de tipo `StatefulWidget`. La forma más sencilla de escribir una subclase de `StatefulWidget` es escribiendo `stful` y pulsar `Enter` y justo a continuación, escribir el nombre de la clase: `ConversorPaginaPrincipal` y pulsar `Enter` otra vez.

Hay algunos errores de compilación que desaparecen con el import:

```
import 'package:flutter/material.dart';
```

El archivo `conversor_pagina_principal.dart` ahora contiene el siguiente código:

```
import 'package:flutter/material.dart';

class ConversorPaginaPrincipal extends StatefulWidget {
  const ConversorPaginaPrincipal({Key? key}) : super(key: key);

  @override
  State<ConversorPaginaPrincipal> createState() => _ConversorPaginaPrincipalState();
}

class _ConversorPaginaPrincipalState extends State<ConversorPaginaPrincipal> {
  @override
  Widget build(BuildContext context) {
    return const Placeholder();
  }
}
```

Veamos algunos aspectos interesantes de este código:

1. No se ha creado una clase, como en el caso de `StatelessWidget` sino que se han creado dos clases, la clase `ConversorPaginaPrincipal` y la clase `_ConversorPaginaPrincipalState` que como su nombre indica es **privada**.

Eso es así, porque una subclase de `StatefulWidget` tiene asociado un estado mutable que cambia durante la ejecución de la aplicación. En el ejemplo del conversor, el estado mutable permitirá almacenar y cambiar los datos de la última conversión realizada. Esto permite a la aplicación responder a las acciones del usuario.

La mejor forma de gestionar esto es separar la gestión del estado en una clase aparte que es subclase de `State`. Dado que las subclases de `State` se declaran para gestionar el trabajo de una subclase de `StatefulWidget` concreta y está muy acoplada con ésta, no tiene sentido que sea una clase pública. Lo mejor es hacerla privada y escribir las dos clases juntas en el mismo archivo.

2. La clase `State` es genérica. Eso significa que para poder utilizarla, hay que escribir entre ángulos `< >` el nombre de la clase con la que está asociado ese estado. En esta aplicación el estado está asociado a la clase `ConversorPaginaPrincipal` por lo que se declara así:

```
State<ConversorPaginaPrincipal>
```

3. La subclase de `StatefulWidget`, `ConversorPaginaPrincipal`, no redefine el método `build`, sino que redefine el método `createState` que no tiene argumentos y que devuelve una subclase de la clase `State<ConversorPaginaPrincipal>`. La subclase que implementa la gestión del estado de la clase `ConversorPaginaPrincipal`.
4. Quien redefine el método `build` es la clase `_ConversorPaginaPrincipalState` que es la clase que gestiona el estado asociado a la clase `ConversorPaginaPrincipal`.

Como en el caso de las subclases de `StatelessWidget` el método `build` recibe como argumento un contexto y devuelve un `Widget`.

Ese método `build` se llama cada vez que el estado mutable cambia, para reflejar el nuevo estado en la interfaz de usuario de la aplicación. Por ejemplo, cada vez que se realiza una conversión, el estado cambia para poder mostrar en la interfaz de usuario el valor calculado.

5. El constructor de `ConversorPaginaPrincipal` es `const` ya que lo que cambia en una vista `StatefulWidget` no es la vista en sí, sino su estado. Por tanto, para aprovechar las ventajas de construir instancias canónicas, el constructor se declara `const`.

Ya que tenemos una página principal para la aplicación, es el momento de conectarlo con ésta. Para ello, en el archivo: `conversor_app.dart` buscaremos la línea:

```
home: const Placeholder(),
```

y la sustituiremos por la siguiente:

```
home: const ConversorPaginaPrincipal(),
```

Diferencia entre widgets sin estado y widgets con estado

Los widgets cuyo estado nunca cambia, no necesitan guardar un estado mutable, ya que su estado no va a cambiar durante toda la ejecución de la aplicación. Estos widgets deben ser subclases de `StatelessWidget`.

Los widgets cuyo estado puede cambiar durante la ejecución de la aplicación deben declararse como subclases de `StatefulWidget` con una clase privada asociada para gestionar el estado.

En resumen:

1. Para declarar una subclase de `StatelessWidget` hay que:
 - i. Crear una clase que extienda a la superclase `StatelessWidget`
 - ii. Sobreescibir el método `build` que recibe como argumento un contexto `BuildContext` y devuelve un `Widget` .
2. Para declarar una subclase de `StatefulWidget` hay que:
 - i. Crear una clase que extienda a la superclase `StatefulWidget`
 - ii. Crear una clase privada que extienda a la superclase genérica `State` usando como valor asociado el de la subclase de `StatefulWidget`
 - iii. Sobreescibir el método `createState` en la subclase de `StatefulWidget` Ese método lo único que debe hacer es construir una instancia del estado asociado y devolverla.
 - iv. Sobreescibir el método `build` en la subclase de `State` Ese método recibe como argumento un contexto `BuildContext` y devuelve un `Widget` .

6 – Completar la página principal de la aplicación

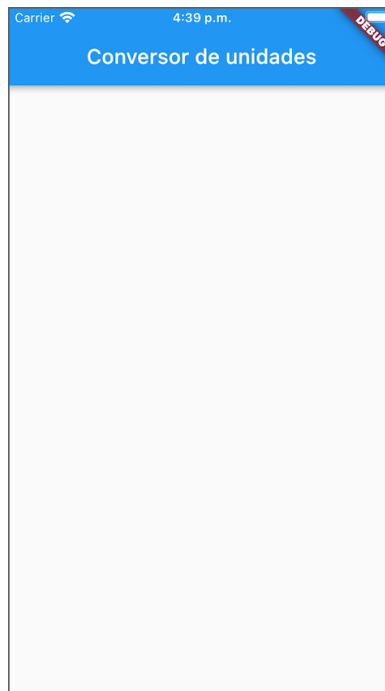
Comenzaremos sustituyendo el método `build` de la clase `_ConversorPaginaPrincipalState` :

```
@override
Widget build(BuildContext context) {
  return const Placeholder();
}
```

por esta otra versión:

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Conversor de unidades'),
    ),
    body: const SafeArea(
      child: Center(),
    ),
  );
}
```

Ahora, si se ejecuta la aplicación, se obtiene una salida que no es una pantalla en negro como hasta ahora:



Aspecto de la aplicación en este paso

La aplicación todavía no está completa, pero antes de seguir veamos algunos aspectos interesantes:

1. En el corazón de toda página principal de una aplicación hay un widget de tipo `Scaffold`. En Inglés, scaffold significa andamio. Es un nombre muy apropiado ya que este widget permite definir fácilmente varias zonas para la ventana. Cada una de esas zonas se define mediante una propiedad con nombre del constructor de `Scaffold`

En esta aplicación hemos usado dos de esas zonas. En próximas sesiones aprenderemos a usar otras zonas definidas por el widget `Scaffold`

2. En esta aplicación se usa la propiedad `appBar` de la vista `Scaffold` para define una barra de aplicación.

Nota: El patrón de interacción llamado: **barra de aplicación**, consiste en un rectángulo de pequeña altura situado en la parte superior de la pantalla, debajo de la barra de estado, que puede incluir además de un título, botones para acceder a las **herramientas más frecuentes de una determinada pantalla**.

A diferencia de los botones de la barra de pestañas, en los botones de la barra de aplicación solo se usan iconos sin texto explicativo. Por lo general, las herramientas se colocan alineadas a la derecha.

En esta aplicación no hay herramientas, por tanto en la barra de aplicación solo se define un título, por eso solo se usa la propiedad `title` de la clase `AppBar` a la que se pasa un widget de la clase `Text`. Los widgets de la clase `Text` muestran texto de salida, es decir, el usuario solo puede leer su contenido, pero no escribir en ellos.

3. **body:** Como su nombre indica, es el cuerpo de la página principal. Recibe como argumento el widget que queramos que ocupe esa zona.

En este ejemplo se ha usado un objeto de la clase `SafeArea` que añade el sangrado necesario para asegurar que las vistas hijas que se coloquen en ese cuerpo no se vean tapadas por elementos del sistema operativo o por las formas de la pantalla del dispositivo como por ejemplo esquinas redondeadas, etc.

La clase `SafeArea` es un **contenedor**, un widget que puede contener a otro widget. Para eso tiene la propiedad `child:` que nos permite indicar qué widget queremos colocar dentro del área segura.

En este caso hemos usado una instancia de la clase `Center` que es un **widget posicional**. También **es un contenedor que centra su contenido en la pantalla**. El widget que se coloque dentro de un widget de tipo `Center` estará centrado horizontal y verticalmente en la pantalla del dispositivo.

Por ahora el widget contenedor `Center` está vacío. Pronto lo llenaremos de contenido.

Vamos a rellenar el cuerpo de la pantalla principal con widgets que permitan realizar la conversión de unidades. Se necesitará un campo en el que escribir el valor a convertir, un desplegable para escoger la conversión que se le quiere aplicar (por ejemplo kilómetros a metros), un botón para realizar la conversión y un texto donde se muestre el resultado. Son varios widgets y lo mejor es situarlos unos sobre otros formando una columna.

En Flutter existe un widget contenedor llamado `Column` cuya misión es justamente esa. Puede albergar en su interior más de un widget hijo, y los coloca en la pantalla, uno debajo del otro formando una columna.

Para ello la clase `Column`, a diferencia de la clase `Center` que tiene la propiedad `child:` y que solo acepta un widget hijo, tiene una propiedad llamada `children:` que recibe una lista de widgets.

Dentro del método `build` de la clase `_ConversorPaginaPrincipalState` buscaremos las líneas:

```
body: const SafeArea(  
  child: Center(),  
)
```

y las sustituiremos por:

```
body: SafeArea(  
  child: Center(  
    child: Column(  
      children: const <Widget>[  
      ],  
    ),  
  ),  
)
```

Por ahora la lista de `Widget` está vacía, la vamos a rellenar a continuación.

Cuidado: La palabra `const` que había justo delante de `SafeArea` ha desaparecido y ahora se coloca justo delante de la lista de `Widget`.

Etiqueta para el valor a convertir

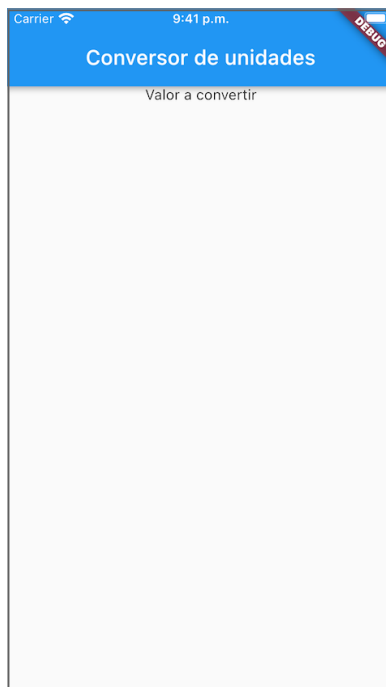
Es una buena idea colocar antes del campo donde escribir el valor a convertir, una etiqueta que indique al usuario cuál es el contenido del campo. Esto le ayudará a entender y usar la interfaz.

Usaremos el texto `'Valor a convertir'`

Para ello usaremos el widget `Text` ya conocido. Dentro de la lista de widgets que se pasa en la propiedad `children:` de la clase `Column` añadiremos lo siguiente:

```
Text('Valor a convertir',),
```

Si hacemos Hot Reload veremos que ahora la interfaz tiene este aspecto:



Aspecto de la aplicación tras añadir la etiqueta para indicar cuál es el contenido del campo de texto

No es exactamente lo que teníamos en mente. Hay dos problemas:

1. La etiqueta está centrada horizontalmente, pero no verticalmente, sino que aparece alineada en la parte superior. El widget `Center` alinea en el centro a su widget hijo, tanto horizontalmente como verticalmente. Entonces, ¿qué es lo que ha fallado?

No ha fallado nada. Recuerda que el widget hijo es `Column` y ese sí que está alineado tanto horizontalmente como verticalmente. El problema es que, por defecto, **la columna ocupa todo el espacio vertical disponible**, así que es tan alta como el cuerpo de la página principal.

Dentro de la columna, por defecto, sus elementos se colocan alineados en la parte superior, así que por eso el texto `Valor a convertir` aparece arriba.

La solución es sencilla. Sólo hay que usar la propiedad `mainAxisAlignment` del widget `Column` para indicar que los elementos de la columna también estén alineados verticalmente.

```
mainAxisAlignment: MainAxisAlignment.center,
```

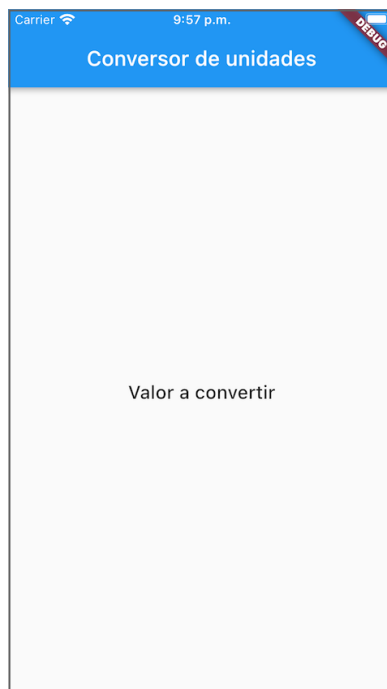
2. El texto es demasiado pequeño. No es legible y puede que algunos usuarios no puedan leerlo con claridad.

La solución también es sencilla. El widget `Text` tiene la propiedad `style` que permite personalizar la forma en que se muestra el texto en la pantalla.

```
Text(  
  'Valor a convertir',  
  style: TextStyle(  
    fontSize: 18.0,  
    fontWeight: FontWeight.w500,  
  ),  
)
```

permite mostrar el texto con tamaño 18 dp y seminegrita.

Si hacemos Hot Reload, la interfaz se ve así:



Aspecto de la aplicación tras modificar la alineación y el grosor de la fuente con la que se muestra el texto

que es lo que queríamos.

Campo de texto donde escribir el valor a convertir

El valor a convertir forma parte del estado mutable de la pantalla principal, por ello, debe haber alguna propiedad en la clase `_ConversorPaginaPrincipalState` donde poder guardar ese valor.

Por ello, al comienzo de la clase `_ConversorPaginaPrincipalState` añadiremos la siguiente propiedad privada:

```
late double _valorAConvertir;
```

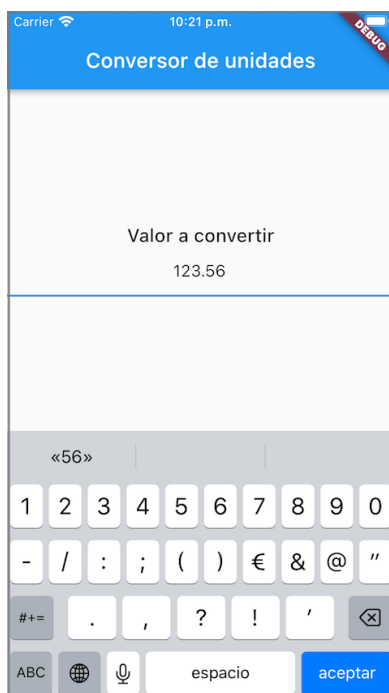
y un nuevo widget a la lista que se pasa a la clase `Column` en la propiedad `children`. Justo debajo del widget `Text` introducido en el paso anterior escribiremos:

```
TextField(  
  textAlign: TextAlign.center,  
  keyboardType: const TextInputType.numberWithOptions(  
    signed: true,  
    decimal: true  
  ),  
  onChanged: (texto) {  
    var valorIntroducido = double.tryParse(texto);  
    if (valorIntroducido != null) {  
      setState(() {  
        _valorAConvertir = valorIntroducido;  
      });  
    } else {  
      setState(() {  
        _valorAConvertir = 0.0;  
      });  
    }  
  },  
)
```

Como vemos, la propiedad `_valorAConvertir` no se ha inicializado y en la clase no hay constructor. El código tal y como está no funciona. Antes de poder ejecutar la aplicación hay que añadir el siguiente método a `_ConversorPaginaPrincipalState`:

```
@override  
void initState() {  
  _valorAConvertir = 0.0;  
  super.initState();  
}
```

Si hacemos Hot Reload el aspecto de la interfaz será parecido a este:



Aspecto de la aplicación tras añadir el cuadro para introducir el valor a convertir

Como vemos la interfaz de usuario es muy usable. El texto que se introduce aparece centrado para una mayor legibilidad y el teclado en pantalla aparece ya preparado para introducir números, lo que simplifica la tarea del usuario.

Veamos algunos aspectos interesantes de ese código:

1. La propiedad `_valorAConvertir` se ha marcado como `late` para que pueda ser inicializada posteriormente en el método `initState`. De otra forma se produce un error de compilación.
2. El método `initState` se llama una sola vez cuando se construye el estado. Aquí es donde generalmente se asignan los valores iniciales para las propiedades. En este caso, estamos estableciendo el valor `0.0` como valor inicial para la propiedad `_valorAConvertir`.

Importante: Si se redefine el método `initState`, siempre se debe llamar a `super.initState();` al final del mismo.

3. Se usa un widget de clase `TextField` que es el widget que se usa en Flutter cuando se quiere proporcionar al usuario un campo donde pueda escribir alguna entrada.
4. Para que el texto que el usuario escribe en el `TextField` aparezca centrado, se usa la propiedad `textAlign:` con el valor `TextAlign.center`. El argumento es de tipo `TextAlign` que es un `enum` que contiene los posibles modos de alinear un texto en un campo de texto.
5. A continuación se usa la propiedad `keyboardType:` de la clase `TextField` para elegir el teclado que se muestra al usuario por defecto. Se ha elegido un teclado numérico con las opciones `signed:` y `decimal:` con valor `true`. Si se hubiese elegido un teclado de tipo `TextInputType.number` solo se podrían introducir enteros.

6. Existen varias formas en Flutter de actuar cuando un usuario escribe algo en un widget `TextField` . La que usamos aquí es mediante la propiedad `onChanged:` que recibe como argumento una función anónima que tiene un único parámetro de tipo `String`

Cada vez que el usuario cambie el valor del campo de texto (escriba, borre, etc.) se ejecuta la función anónima que se pasa a `onChanged:` Esa función tiene un argumento de tipo `String` que aquí se ha llamado `texto` . A través de él se tiene acceso al texto tecleado por el usuario en el `TextField`

El usuario puede escribir, por equivocación, algo que no sea un número, por eso antes de usar el valor contenido en la variable `texto` hay que parsearlo para convertirlo en un `double` . Eso se hace con el método `tryParse` de la clase `double` que devuelve un valor de tipo `double?` Si el valor introducido por el usuario no corresponde con un número válido, entonces se devuelve `null`

En la variable `valorIntroducido` de clase `double?` se tiene, o bien `null` si el valor no es correcto o bien un `double` . Con él se actualizará el estado de la clase

`_ConversorPaginaPrincipalState` representado por la propiedad `_valorAConvertir`

Para actualizar el estado, debe llamar al método `setState` de la clase `State` que se hereda en `_ConversorPaginaPrincipalState` .

El método `setState` le dice a Flutter que el estado de un objeto ha cambiado y que la interfaz de usuario debe actualizarse. Dentro del método `setState` , se cambian las propiedades de la clase que almacenen el estado mutable (en nuestro caso, `_valorAConvertir`).

Muy importante: Si se quiere modificar alguna propiedad que forma parte del estado mutable de una subclase de la clase `State` hay que hacerlo dentro de una función anónima que se pasa al método `setState` de la clase `State` . De lo contrario Flutter no sabrá que se ha cambiado el estado y no refrescará la interfaz.

Añadir un desplegable para elegir el tipo de conversión

Es el momento de añadir los componentes de la interfaz para mostrar al usuario las distintas conversiones que se pueden realizar. Para ello, añadiremos las siguientes propiedades a la clase `_ConversorPaginaPrincipalState` :

```
var _conversiones = ['Kilómetros a metros', 'Metros a kilómetros'];  
late int _idConversionActual;
```

añadiremos la siguiente inicialización al método `initState` justo antes de la llamada a `super.initState();` :

```
_idConversionActual = 0;
```

y escribiremos dos nuevos widgets en la lista de widgets que se pasa al widget de clase `Column` , justo detrás del widget `TextField` que añadimos en el paso anterior:

```
const Text(  
  'Convertir de',  
  style: TextStyle(  
    fontSize: 18.0,  
    fontWeight: FontWeight.w500,  
  ),  
) ,  
DropdownButton<String>(  
  value: _conversiones[_idConversionActual],  
  items: _conversiones.map((String value) {  
    return DropdownMenuItem<String>(  
      value: value,  
      child: Text(value),  
    );  
  }).toList(),  
  onChanged: (nuevoValor) {  
    setState(() {  
      _idConversionActual = _conversiones.indexOf(nuevoValor!);  
    });  
  },  
) ,
```

Si hacemos Hot Reload vemos que ahora la interfaz de usuario tiene este aspecto:



Aspecto de la aplicación tras añadir el desplegable para elegir la conversión que se quiera aplicar

La interfaz es más completa, pero los componentes aparecen amontonados unos sobre otros. Visualmente no queda muy bien. Lo cambiaremos para que tenga un aspecto más cuidado y profesional, pero antes veamos algunos aspectos interesantes del código que acabamos de añadir:

1. Las conversiones posibles se tienen predefinidas en una lista privada de `String` llamada `_conversiones` . Por ahora solo se contemplan dos posibles conversiones para nuestra aplicación, convertir de kilómetros a metros y de metros a kilómetros.

En una sección posterior añadiremos nuevas conversiones.

Atención spoiler: Esta no es la mejor forma de codificar esto, pero nos dará juego para aprender en la práctica a aplicar un principio de diseño fundamental, el OCP.

2. La conversión seleccionada en un momento determinado se almacena en la propiedad privada `_idConversionActual`. Es de tipo `int` y representa al índice de la conversión seleccionada. Posteriormente en el método `initState` se le asigna el valor inicial `0`.
3. Se ha añadido un widget de clase `Text` con el texto `Convertir de` que sirve de título y ayuda a comprender al desplegable. Las opciones utilizadas son las mismas que se han usado para la etiqueta `Valor a convertir`, si es necesario se pueden consultar ahí.
4. A continuación se usa el widget `DropDownButton` de Flutter para construir el desplegable. Es un widget que permite a los usuarios seleccionar un valor de una lista de valores. En todo momento muestra el valor seleccionado actualmente, así como un pequeño triángulo que despliega una lista para seleccionar otro valor.



Aspecto del desplegable asociado a un widget `DropDownButton`

En este widget llaman la atención algunas cosas:

- i. Es genérico, lo que permite que el desplegable contenga valores de varias clases. En este caso se ha usado el tipo `String`. Eso indica que cuando el usuario seleccione un nuevo valor en el desplegable el tipo asociado a ese valor seleccionado será `String`.
- ii. El valor asociado, el que se muestra seleccionado en el desplegable, se indica mediante la propiedad `value:`. En este caso se pasa el valor `_conversiones[_idConversionActual]`.
`_idConversionActual` forma parte del estado mutable de la clase, por lo que cuando cambie, se refrescará la interfaz y en consecuencia el valor mostrado en el desplegable también cambiará.
- iii. La lista que muestra los valores del desplegable es de tipo `List<DropDownMenuItem<String>>?`. Es decir, es una lista anulable de widgets de tipo `DropDownMenuItem<String>`. Eso es lo que hay que pasarle a la propiedad `items:` de la clase `DropDownButton`.
- iv. En nuestro caso, las posibles conversiones están en una lista, pero de `String`, así que hay que transformarla en una lista de `DropDownMenuItem<String>` antes de poder pasarla a la propiedad `items:`.

Esto puede hacerse con la ayuda del higher-order method `map` de la clase `List` aplicado a la lista `_conversiones`. Como sabemos `map` recibe como argumento una función anónima con un parámetro del mismo tipo que los elementos de la lista a la que se aplica, en este caso la lista `_conversiones`, cuyos argumentos son de tipo `String`. La función anónima se ejecuta una vez para cada elemento y devuelve un `Iterable` formado por el resultado de aplicar la función anónima a cada elemento de la lista.

En este caso lo único que hay que hacer en el interior de la función anónima es construir un `DropDownMenuItem` con el `String` que se recibe como argumento y devolverlo.

- v. El higher-order method `map` devuelve un `Iterable`, pero lo que se debe pasar a la propiedad `items:` de la clase `DropDownButton` es una instancia de la clase `List`.

Recordemos que la clase `Iterable` tiene un método llamado `toList` que permite hacer la conversión de forma sencilla. Solo hay que invocarlo detrás del paréntesis de cierre del método `map`

- vi. Al igual que en el caso del widget `TextField`, se usa la propiedad `onChanged:` para responder cuando el usuario seleccione un nuevo valor en el desplegable. Esta propiedad recibe una función anónima con un único argumento de tipo `String?` que contiene el nuevo valor seleccionado por el usuario.

Dentro de la función anónima hay que actualizar el valor de `_idConversionActual` que es parte del estado mutable, así que hay que hacerlo dentro del método `setState` que de nuevo recibe una función anónima sin argumentos.

En el cuerpo de esa función anónima se busca si en la lista `_conversiones` hay algún valor que coincida con el `nuevoValor` y se usa su índice para actualizar el valor de `_idConversionActual`

Al escribirla hay que tener en cuenta que el tipo de `nuevoValor` es `String?` por lo que debe usarse así: `nuevoValor!`

Ahora veamos algunas de las alternativas que tenemos para que los elementos no aparezcan tan juntos y haya un poco de espacio alrededor de la pantalla principal.

- Se puede mejorar el aspecto de una vista controlando la separación entre sus bordes y los del contenedor en el que está situada. Para ello Flutter proporciona el widget contenedor `Padding` que añade un espacio alrededor de su widget hijo. El relleno puede aplicarse a todos los lados por igual o solo a algunos de ellos.

Vamos a colocar el widget `Column` dentro de un widget `Padding`. Este widget tiene un solo widget hijo, y lo dibuja en la pantalla dejando un poco de espacio alrededor.

El árbol de vistas de nuestra aplicación, en este momento, es complejo. Así que no es sencillo insertar un nuevo elemento alrededor de otro ya construido manualmente.

Sin embargo, Android Studio y su plugin de Flutter nos ayudan en esta tarea:

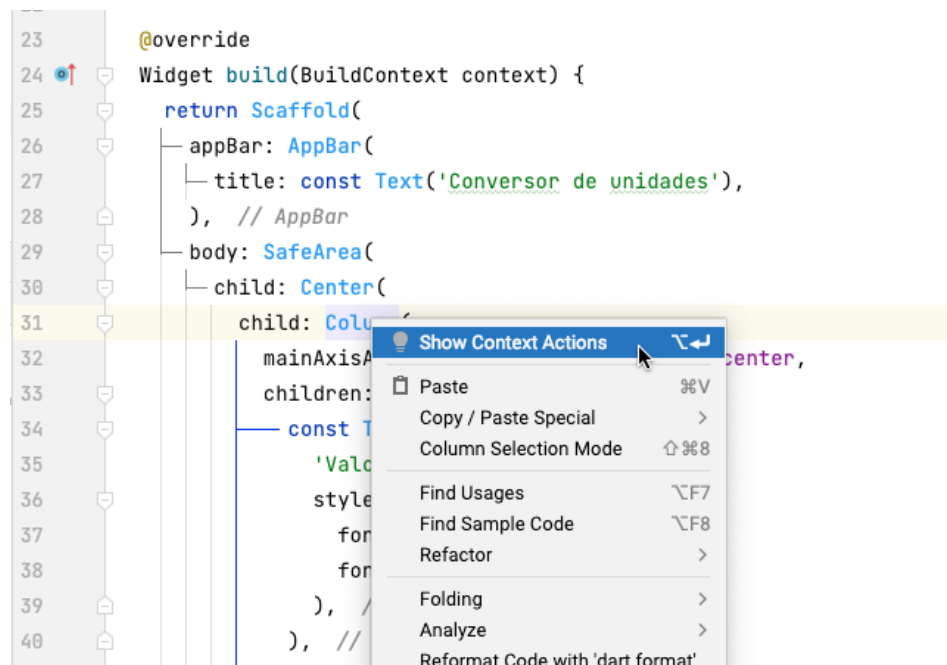
- i. En primer lugar, haremos clic con el ratón sobre la llamada al constructor de la clase

`Column` :



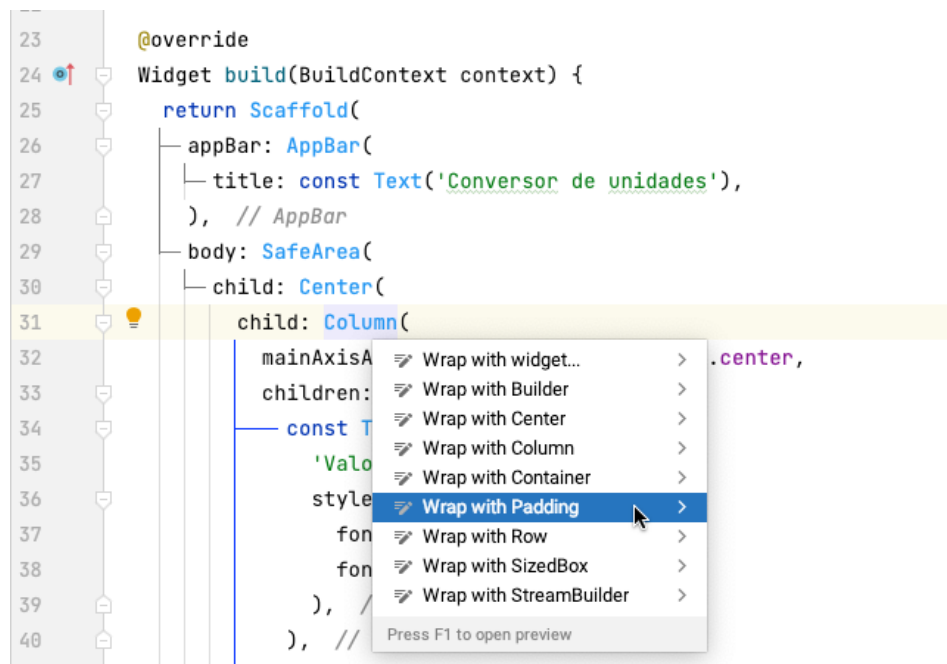
Paso 1: Clic con el ratón sobre la llamada al constructor de la clase `Column`

- ii. A continuación, haremos clic derecho para que se muestre el menú de contexto, y dentro de éste elegiremos la primera opción `Show Context Actions` :



Paso 2: Clic derecho para mostrar el menú de contexto y en él elegir `Show Context Actions`

iii. En esa lista de acciones elegir `Wrap with padding` :



Paso 3: Escoger la acción `Wrap with padding`

iv. El código se reescribe y el widget `Column` ahora aparece como widget hijo de un widget de clase `Padding` . Solo hay que escoger el valor para la propiedad `padding` : que se quiera. En este caso se ha elegido un valor de `EdgeInsets.all(16.0)` que añade 16 pixels de espacio alrededor del widget `Column` por los cuatro lados:



Paso 4: Escoger el valor para la propiedad `padding` :

- Para dejar espacio entre dos widgets en una lista, una opción muy útil es usar un widget `SizedBox` vacío. Este widget se coloca entre los dos widgets que se quieren separar. La ventaja de este contenedor es que se puede asignar un tamaño concreto, tanto en anchura, como en altura, lo que permite controlar de forma muy precisa la separación entre las vistas.

Buscaremos en el método `build` de la clase `_ConversorPaginaPrincipalState` las líneas:

```
), // TextField
const Text(
```

y las sustituiremos por:

```
), // TextField
const SizedBox(
  height: 32.0,
),
const Text(
```

Tras hacer Hot Reload, ahora el aspecto de la aplicación es más atractivo visualmente:



Aspecto de la aplicación tras añadir espacio alrededor y separación entre los widgets `TextField` y `Text`

Medidas independientes del dispositivo

En Flutter, cuando hablamos de píxeles, en realidad estamos hablando de **píxeles independientes de la densidad o dp** (density-independent pixels), y no de píxeles físicos. Los píxeles físicos son el número real de píxeles que tiene un dispositivo. Pero hay varios factores de forma, y la resolución de una pantalla puede variar sustancialmente. Por ejemplo, el Sony Xperia E4 tiene un tamaño de pantalla de 5" y una resolución de 960 * 540 píxeles. El Xperia X tiene el mismo tamaño de pantalla de 5", pero una resolución de 1920 * 1080. Por lo tanto, si quisiéramos dibujar un cuadrado de 540 píxeles por lado, sería mucho más pequeño en el segundo dispositivo.

Es por eso que las medidas se expresan en píxeles independientes de la densidad, dp, lo que permite que los widgets de la interfaz de usuario se vean de igual tamaño en todos los dispositivos, con independencia de su resolución.

Veamos como funciona. Para ello necesitamos estudiar algunos conceptos básicos:

Densidad de píxeles de una pantalla o Screen density

El número de píxeles que caben en una pulgada se conoce como densidad de píxeles.

Las pantallas de alta densidad tienen más píxeles por pulgada que las de baja densidad. Como resultado, los elementos de la interfaz de usuario de las mismas dimensiones de píxeles aparecen más grandes en pantallas de baja densidad y más pequeños en pantallas de alta densidad.

Para calcular la densidad de píxeles de una pantalla se usa esta ecuación:

$$screen\ density = \frac{Anchura\ de\ pantalla\ en\ pixeles}{Anchura\ de\ pantalla\ en\ pulgadas}$$

La independencia de la densidad se refiere a la visualización uniforme de los elementos de la interfaz de usuario en pantallas con diferentes densidades.

Píxeles independientes de la densidad o Density-independent pixels

Los píxeles independientes de la densidad, escritos como dp o dip, son unidades flexibles que se escalan para tener dimensiones uniformes en cualquier pantalla. Proporcionan una forma flexible de mostrar un mismo diseño en todas las plataformas.

Las interfaces de usuario Material Design utilizan píxeles independientes de la densidad para mostrar elementos de manera consistente en pantallas con diferentes densidades.

Cuando se desarrolla una aplicación usando Material Design, se usan dp para mostrar elementos de manera uniforme en pantallas con diferentes densidades.

Un dp es igual a un píxel físico en una pantalla con una densidad de 160.

Para calcular los dp en pantallas con otras densidades de píxeles se usa esta fórmula:

$$dp = \frac{(dimension\ en\ pixeles * 160)}{screen\ density}$$

Anchura pantalla (pulgadas)	Screen density	Anchura pantalla (píxeles)	Anchura pantalla (dps)
1.5 in	120	180 px	240 dp
1.5 in	160	240 px	240 dp
1.5 in	240	360 px	240 dp

Algunos ejemplos que muestran la relación entre tamaño físico de la pantalla, densidad de píxeles, tamaño en píxeles de la

Los **píxeles escalables o sp**, cumplen la misma función que los píxeles independientes de la densidad, dp, pero para los tipos de letra. El valor predeterminado de un sp es el mismo que el valor predeterminado de un dp.

La principal diferencia entre sp y dp es que los sp preservan la configuración del tipo de letra que haya establecido el usuario. Los usuarios que tengan una configuración de texto más grande para una mejor accesibilidad verán que los tamaños de fuente coinciden con sus preferencias de tamaño de texto.

Añadir el botón para iniciar los cálculos

En la clase `_ConversorPaginaPrincipalState` se necesita una nueva propiedad donde se almacene el resultado de la conversión. Añadiremos a la clase la siguiente propiedad:

```
late double _valorConvertido;
```

que se inicializa en el método `initState`. Añadimos justo delante de la llamada a `super.initState()` lo siguiente:

```
_valorConvertido = 0.0;
```

`_valorConvertido` es parte del estado mutable de la aplicación, es decir, cada vez que se cambie el valor de la propiedad `_valorConvertido` se debe refrescar la interfaz para que se muestre ese resultado al usuario.

A continuación escribiremos los siguientes widgets en la lista de widgets para la columna, justo debajo del `DropDownButton` que escribimos en el paso anterior:

```
const SizedBox(  
  height: 32.0,  
)  
,  
const Text(  
  'Resultado',  
  style: TextStyle(  
    fontSize: 18.0,  
    fontWeight: FontWeight.w500,  
  ),  
)  
,  
const SizedBox(  
  height: 16.0,  
)  
,  
Text(  
  '$_valorConvertido',  
  style: const TextStyle(  
    fontSize: 32.0,  
  ),  
)  
,  
const SizedBox(  
  height: 32.0,  
)
```

```
ElevatedButton(
  child: const Text(
    'Convertir',
    style: TextStyle(
      fontSize: 20.0,
      color: Colors.white,
    ),
  ),
  onPressed: () {
  },
),
```

si hacemos Hot Reload vemos que se han añadido los widgets que faltaban a la interface.



Aspecto de la aplicación tras añadir el resto de widgets

De todos los widgets que se han añadido, el único que es nuevo es `ElevatedButton`. Veamos algunos aspectos interesantes de este widget.

1. Un `ElevatedButton` es un widget que facilita la interacción con el usuario. Se muestra resaltado para atraer la atención del usuario y cuando éste lo toca, se redibuja con una animación que indica que realmente se ha pulsado.

Implementa de forma muy sencilla para el programador, la doble realimentación que debe usarse en todas las interfaces de usuario para aplicaciones móviles:

- El primer nivel de realimentación consiste en informar claramente al usuario de que realmente ha pulsado el elemento de interacción correspondiente (en este caso el botón)
- El segundo nivel de realimentación consiste en mostrar claramente al usuario el resultado de la acción desencadenada por el elemento de interacción correspondiente.

2. Entre sus propiedades destacan dos:

- i. `child`: que permite indicar cuál es el contenido del botón. Lo normal es que se use como hija una vista de tipo `Text` para escribir un texto dentro del botón, pero también se podían usar otras vistas para añadir iconos u otra información.

En este caso se ha usado una etiqueta con el texto: `Convertir` y un estilo para ajustar su tamaño y color.

- ii. `onPressed`: En esta propiedad se pasa una función anónima. La que queremos que se ejecute cuando el usuario pulse el botón. Por ahora se ha dejado en blanco, porque todavía no se ha programado el modelo de la aplicación (las clases que implementan la lógica de negocio de la aplicación). Volveremos sobre este punto un poco más adelante.

Nota: También existe la propiedad `onLongPress` que permite distinguir cuando el usuario hace una pulsación prolongada sobre el botón.

3. Si no se indica un estilo concreto para el botón, su color de fondo, etc. se toman del tema de la aplicación.

Nota: En la próxima sesión de prácticas estudiaremos cómo programar un tema personalizado.

4. Si no se asigna ninguna función anónima ni a la propiedad `onPressed` ni a la propiedad `onLongPress`, entonces toman por defecto el valor `null` y el botón se muestra deshabilitado, es decir, se muestra en color claro y no se puede pulsar.

7 – Probando la interfaz de usuario

Ya está terminada la interfaz de usuario de la aplicación. Todavía no se ha escrito el modelo de la aplicación, por lo que no hace ninguna tarea, pero antes de seguir, conviene probar a fondo esa interfaz para detectar de forma temprana posibles problemas.

Cuando hablo de probar a fondo la interfaz me refiero a probarla en varios dispositivos con diferentes características, por ejemplo, diferentes tamaños de pantalla. Para eso habremos generado varios emuladores con características físicas distintas.



En esta imagen puede verse, como la interfaz cabe perfectamente en un dispositivo con pantalla grande (izquierda), aunque se esté usando el teclado, sin embargo, en un dispositivo con pantalla más pequeña (derecha) hay elementos que no se pueden acceder cuando se usa el teclado

Si hay algún problema y no todos los elementos de la interfaz caben en la pantalla actual, Flutter lo muestra mediante un patrón visual de rayas amarillas y negras sobre las que se muestra un mensaje. En este ejemplo:

Bottom overflowed by 58 pixels

Flutter proporciona una solución sencilla para este problema. Consiste en incluir las vistas cuyo tamaño excede el de la pantalla dentro de un widget de tipo `SingleChildScrollView`. Como su nombre indica es un contenedor que tiene un único widget hijo y que permite hacer scroll para acceder a todos los componentes de ese widget hijo, aunque no quepan todas a la vez en la pantalla. En nuestro ejemplo, incluiremos el widget `Padding` dentro de un widget `SingleChildScrollView`

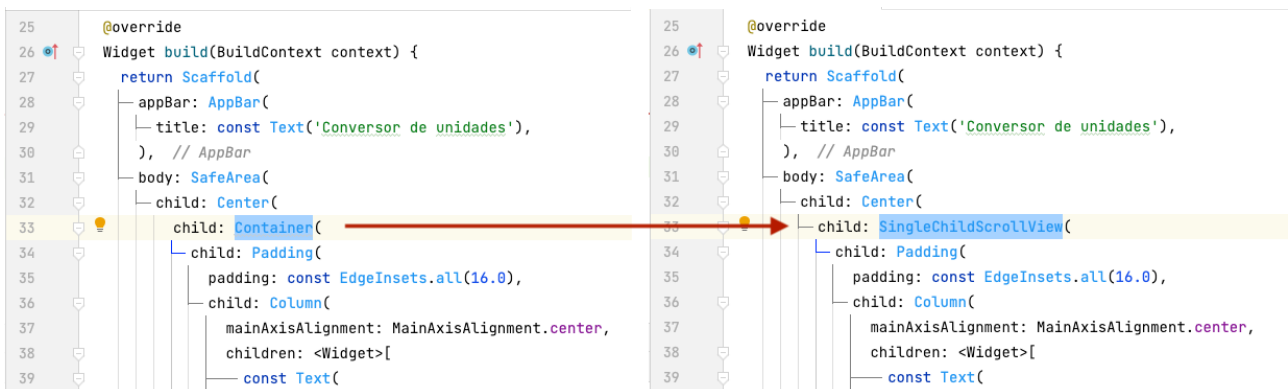
Anteriormente, tuvimos que incluir la vista `Column` dentro de una vista `Padding` para dejar espacio alrededor, y vimos que la forma más sencilla de hacerlo era mediante las **acciones de contexto** `Context Actions` que proporciona el plugin de Flutter para Android Studio.

Sin embargo, si hacemos clic sobre la llamada al constructor de la clase `Padding`, a continuación hacemos clic derecho para mostrar el menú de contexto y ahí elegimos el menú `Show Context Actions`, vemos como no existe ninguna acción de contexto para envolver con una vista `SingleChildScrollView`



Entre las acciones de contexto no hay ninguna etiquetada como `wrap with SingleChildScrollView` >

Pero no es problema ya que podemos envolver la vista `Padding` con una vista de tipo `Container` y después sustituir a mano la palabra `Container` por `SingleChildScrollView`



Sin embargo puede usarse `wrap with Container` > y a continuación cambiar a mano `Container` por `SingleChildScrollView`

Consejo: Siempre debes probar tus interfaces en diferentes tamaños de pantalla y orientaciones para comprobar que en todos los casos se ve correctamente.

Con esto el código del método `build` de la clase `_ConversorPaginaPrincipalState` queda así:

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar( title: const Text('Conversor de unidades'), ),
    body: SafeArea(
      child: Center(
        child: SingleChildScrollView(
          child: Padding(
            padding: const EdgeInsets.all(16.0),
            child: Column(
              mainAxisAlignment: MainAxisAlignment.center,
              children: <Widget>[
                const Text(
                  'Valor a convertir',
                  style: TextStyle(
                    fontSize: 18.0,
                    fontWeight: FontWeight.w500,
                  ),
                ),
                TextField(
                  textAlign: TextAlign.center,
                  keyboardType: const TextInputType.numberWithOptions(
                    signed: true, decimal: true),
                  onChanged: (texto) {
                    var valorIntroducido = double.tryParse(texto);
                    if (valorIntroducido != null) {
                      setState(() => _valorAConvertir = valorIntroducido );
                    } else {
                      setState(() => _valorAConvertir = 0.0 );
                    }
                  },
                ),
                const SizedBox( height: 32.0, ),
                const Text(
                  'Convertir de',
                  style: TextStyle(
                    fontSize: 18.0,
                    fontWeight: FontWeight.w500,
                  ),
                ),
                DropdownButton<String>(
                  value: _conversiones[_idConversionActual],
                  items: _conversiones.map((String value) {
                    return DropdownMenuItem<String>(
                      value: value,
                      child: Text(value),
                    );
                  }).toList(),
                  onChanged: (nuevoValor) {
                    setState() {
                      _idConversionActual = _conversiones.indexOf(nuevoValor!);
                    }
                  },
                ),
              ],
            ),
          ),
        ),
      ),
    ),
  );
}
```

```

const SizedBox( height: 32.0, ),
const Text(
  'Resultado',
  style: TextStyle(
    fontSize: 18.0,
    fontWeight: FontWeight.w500,
  ),
),
const SizedBox( height: 16.0, ),
Text(
  '$_valorConvertido',
  style: const TextStyle(
    fontSize: 32.0,
  ),
),
const SizedBox( height: 32.0, ),
ElevatedButton(
  child: const Text(
    'Convertir',
    style: TextStyle(
      fontSize: 20.0,
      color: Colors.white,
    ),
  ),
  onPressed: () {},
),
],
),
),
),
),
),
),
);
}

```

8 – Escribir el modelo de la aplicación

Ya que se ha probado que la interfaz de la aplicación está completa y se ve bien en diferentes tamaños de pantalla, es el momento de terminar la aplicación móvil. Para ello escribiremos el modelo de la aplicación.

Esta aplicación es tan simple, que no escribiremos una clase específica para gestionar el modelo, sino que añadiremos esta responsabilidad a la clase `_ConversorPaginaPrincipalState`.

Al final de la clase `_ConversorPaginaPrincipalState` añadiremos el siguiente método privado:

```
void _convertir() {
    double constanteConversion = 1.0;
    switch(_idConversionActual) {
        case 0: // Kilómetros a metros
            constanteConversion = 1000.0;
            break;
        case 1: // Metros a kilómetros
            constanteConversion = 0.001;
            break;
        default:
            break;
    }
    setState(() {
        _valorConvertido = _valorAConvertir * constanteConversion;
    });
}
```

Lo más importante de este método es acordarse de modificar el valor de la propiedad `_valorConvertido` dentro de una función anónima que se pasa al método `setState` de la clase `State`. Si no se hace así, aunque se modifique el valor de la propiedad `_valorConvertido`, la interfaz no se refrescará y por tanto se seguirá mostrando el valor antiguo.

Por último, hay que conectar el método `_convertir` con la interfaz de usuario. En concreto hay que conseguir que al pulsar el botón `Convertir` se llame a este método.

El proceso es sencillo. Solo hay que buscar la línea:

```
onPressed: () {},
```

dentro de `ElevatedButton` y sustituirla por la línea:

```
onPressed: () => _convertir(),
```

Con ello la aplicación está completamente terminada y es funcional. Podemos probarla haciendo algunas conversiones.



Aspecto de la aplicación terminada, completamente funcional

¿Es realmente una buena aplicación?

Como estudiamos en la sesión 3: Principios y patrones de diseño de software, que una aplicación haga lo que su usuario necesita y que se haya programado usando programación orientada a objetos, son solo dos de los pilares que se necesitan para que una aplicación sea una buena aplicación.

El tercer pilar es comprobar cómo de buena es la aplicación aceptando los cambios que sin duda se van a producir a lo largo de su vida.

Atención spoiler: La aplicación, tal y como está no es buena. No soporta bien los cambios. Ya has aprendido las técnicas y patrones de diseño que te permiten corregir estas situaciones anómalas. Es el momento de que las pongas en práctica. Para ello, resuelve los ejercicios propuestos.

Mini ejercicios

1. ¿Por qué crees que el subtítulo de esta sesión de prácticas es: Una aplicación con vistas? Explica con detalle tu respuesta.
2. ➡ Analiza la aplicación de conversión. Se ha escrito usando programación orientada a objetos, pero ¿se han usado correctamente las técnicas de programación orientada a objetos, o se incumple alguno de los principios de diseño estudiados en la sesión anterior? Refactoriza la aplicación para que ya no se incumpla.
3. ➡ Añade otra conversión de tu elección a la aplicación. Explica los pasos que has tenido que seguir. ¿Se ha incumplido alguno de los principios estudiados en la sesión 3 al realizar este cambio? ¿Por qué se ha incumplido? Refactoriza la aplicación para que ya no se incumpla. A continuación añade una nueva conversión para probar cuántos pasos son necesarios ahora.
4. ➡ Después de haber terminado los ejercicios anteriores tendrás una buena aplicación móvil, es decir, que realiza correctamente las conversiones, que ha usado correctamente las técnicas de programación orientada a objetos y que permite hacer cambios de forma sencilla. Ahora es el momento de ir más allá y ver si la experiencia de uso es la mejor.

¿Crees que el botón `Convertir` es realmente necesario? ¿La aplicación sería más o menos usable sin él? Justifica tu respuesta.
5. ➡ Rediseña la aplicación para que no sea necesario el botón `Convertir`. Programa la aplicación rediseñada sin ese botón. ¿Qué problemas has encontrado? ¿Cómo los has resuelto?
6. ¿Qué conclusiones sobre usabilidad podemos obtener de los dos ejercicios anteriores? ¿Y sobre diseño y programación? Justifica tus respuestas.